ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

TIỂU LUẬN MÔN HỌC
IT5440 – NGUYÊN LÝ VÀ KỸ THUẬT PHÂN TÍCH CHƯƠNG TRÌNH
(Principle and Technique of Program Analysis)

Đề tài 02-KH – Phát hiện lỗi và định vị lỗi (hướng nghiên cứu khoa học)

GROUNDED OR GUESSED: TÁI HIỆN VÀ ĐÁNH GIÁ VIỆC ĐỊNH VỊ LỖI NGỮ NGHĨA BẰNG MÔ HÌNH NGÔN NGỮ LỚN

Giảng viên hướng dẫn: PGS. TS. Huỳnh Quyết Thắng

Nhóm M9:
Doãn Văn An – 20252778M (nhóm trưởng)
Tạ Công Nam – 20252640M
Nguyễn Hoàng Hải – 20252583M
Đỗ Xuân Tráng – 20252774M
Hoàng Trung Kiên – 20251333M
Mai Văn Tùng – 20252788M
Nguyễn Đức Tùng – 20252772M

Học kỳ: 20261

Mã nguồn và dữ liệu: `https://github.com/doanan293/grounded-or-guessed`

HÀ NỘI – 2026

Cam kết về tính trung thực

Nhóm M9 cam kết nội dung tiểu luận là kết quả làm việc của nhóm dưới sự hướng dẫn của PGS. TS. Huỳnh Quyết Thắng. Mọi số liệu trong báo cáo được sinh tự động từ mã nguồn và dữ liệu công khai tại `https://github.com/doanan293/grounded-or-guessed`; mỗi con số trong văn bản là một macro được tạo từ tệp dữ liệu tương ứng, không gõ tay.

Phương pháp SemLoc, mã nguồn gốc và bộ dữ liệu SemFault-250 thuộc về nhóm tác giả của bài báo gốc [1] và được dùng theo giấy phép Apache-2.0; mọi phần lấy từ SemLoc đều được ghi nguồn và không được trình bày như đóng góp của nhóm. Hai bài báo còn lại do giảng viên giao [2, 3] và các tài liệu khác được trích dẫn tại chỗ sử dụng.

Theo hướng dẫn của môn học, nhóm có sử dụng công cụ trí tuệ nhân tạo để hỗ trợ đọc tài liệu, viết mã nguồn, chạy thực nghiệm và soạn thảo; mọi kết quả đã được nhóm kiểm tra lại bằng mã nguồn và dữ liệu đi kèm. Riêng bộ nhãn đã rà soát: một LLM rà trước toàn bộ nhãn, sau đó các thành viên nhóm duyệt lại các nhãn bị dời hoặc bị nghi sai trong khi có xem gợi ý của LLM; quy trình và mọi câu trả lời được công bố kèm mã nguồn.

Hà Nội, ngày tháng 11 năm 2026

Đại diện nhóm M9 (nhóm trưởng)

Doãn Văn An

Mục lục

Cam kết về tính trung thực i

Danh mục từ viết tắt và thuật ngữ ix

1 Mở đầu 1

1.1 Đặt vấn đề 1

1.2 Câu hỏi nghiên cứu 2

1.3 Đóng góp chính 2

1.4 Cấu trúc báo cáo 4

2 Tổng quan tài liệu và từ khóa 5

2.1 Phương pháp tìm và chọn tài liệu 5

2.1.1 Nguồn và tiêu chí chọn 5

2.1.2 Bộ từ khóa 5

2.2 Ba bài báo được giao 6

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được . . . 6

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy 7

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp 8

2.3 Các hướng nghiên cứu liên quan 8

2.3.1 Định vị lỗi dựa trên phổ và đột biến 8

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn 9

2.3.3 Thực tế phát triển phần mềm 9

2.3.4 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá 10

2.3.5 Bản sửa vượt kiểm thử nhưng không đúng 10

2.3.6 Nhiễm dữ liệu và tái hiện khi dùng LLM 11

2.4 So sánh và khoảng trống nghiên cứu 11
2.4.1 Bảng so sánh 11
2.4.2 Khoảng trống và hướng của nhóm 12

3 Cơ sở lý thuyết **14**
3.1 Định vị lỗi dựa trên phổ độ phủ 14
3.1.1 Ma trận độ phủ và các công thức nghi ngờ 14
3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm 14
3.2 Lỗi ngữ nghĩa 15
3.2.1 Vì sao độ phủ mất tín hiệu 15
3.2.2 Định nghĩa khách quan bằng độ phủ 15
3.3 SemLoc: phổ vi phạm ràng buộc ngữ nghĩa 16
3.3.1 Quy trình bốn bước 16
3.3.2 Điểm thưởng cho dòng LLM tự đoán 16
3.4 Phân tích tĩnh và phân tích động 17
3.5 Công cụ thống kê 17
3.5.1 Kiểm định cặp 17
3.5.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley 18

4 Phương pháp và thiết kế thực nghiệm **19**
4.1 Câu hỏi nghiên cứu và giả thuyết 19
4.1.1 Ba câu hỏi nghiên cứu 19
4.1.2 Giả thuyết và tiêu chí bác bỏ 19
4.2 Dữ liệu 20
4.2.1 SemFault-250 và cách chia tập 20
4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi . 20
4.2.3 SemFault-250 đã rà soát: LLM rà trước, nhóm duyệt 21
4.2.4 Lỗi thật BugsInPy 22
4.2.5 Biến thể đổi tên định danh 22

4.3 Các phương pháp so sánh 23
4.3.1 Các phương pháp có sẵn 23
4.3.2 LLM + kiểm thử thất bại: phương pháp đề xuất 23
4.3.3 Ablation: tách độ phủ khỏi kiểm thử thất bại 23
4.3.4 Các cách bổ sung 24
4.3.5 Phân tích tĩnh 25
4.4 Kiểm tra cách đánh giá và phân rã đóng góp 25
4.4.1 Kiểm tra cách đánh giá của SemLoc 25
4.4.2 Phân rã "grounded hay guessed" 25
4.5 Thiết lập thực nghiệm 26
4.5.1 Mô hình và hạ tầng 26
4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê 26

5 Kết quả thực nghiệm **27**
5.1 RQ1 – Tái hiện và kiểm tra cách đánh giá 27
5.1.1 Tái hiện và hai cách chấm điểm 27
5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố 28
5.1.3 Chất lượng nhãn của bộ dữ liệu 29
5.2 RQ2 – LLM thật sự cần gì? 30
5.2.1 Một prompt có kiểm thử thất bại so với quy trình nhiều bước . . 30
5.2.2 Thông tin nào tạo ra khác biệt 30
5.2.3 Các cách bổ sung không cải thiện được 31
5.2.4 Phân tích tĩnh 32
5.2.5 Grounded hay guessed: giá trị biên và phân rã 32
5.2.6 Chi phí theo mô hình 33
5.3 RQ3 – Độ bền 34
5.3.1 Lỗi thật BugsInPy 34
5.3.2 Đổi tên định danh 35
5.3.3 Các họ mô hình 36

5.3.4 Ba bộ nhãn 37
5.4 Tóm tắt trả lời các câu hỏi nghiên cứu 37

6 Phân tích thất bại và thảo luận 38
6.1 Phân nhóm tự động 38
6.1.1 Sáu nhóm kết quả 38
6.1.2 Cách chọn ca để đọc 39
6.2 Mơ hồ dòng kề cạnh 39
6.2.1 Dòng đoán lệch có hệ thống xuống dưới 39
6.2.2 Quan hệ đầu khối – thân khối và độ nhạy của Acc@1 39
6.3 Năm mẫu nguyên nhân 40
6.3.1 Dòng điều kiện và câu lệnh được bảo vệ 40
6.3.2 Khối lỗi nhiều dòng 40
6.3.3 Bộ phán định chấp nhận bản sửa bù trừ 40
6.3.4 Nguyên nhân và triệu chứng trên cùng luồng dữ liệu 41
6.3.5 Dòng lỗi chuẩn đáng ngờ 41
6.4 Thảo luận 41
6.4.1 Đọc “guessed” cho đúng 41
6.4.2 Khi nào bằng chứng thực thi có ích 42
6.4.3 Vì sao kiểm thử không làm trọng tài tốt cho vị trí lỗi 42
6.4.4 Bộ dữ liệu bão hòa khi nhãn đúng 43
6.4.5 Hàm ý cho công cụ trong thực tế 43

7 Đóng góp, triết lý khoa học và các mối đe dọa 44
7.1 Các đóng góp khoa học 44
7.1.1 C1. Hạng 1 của SemLoc là dòng LLM đoán 44
7.1.2 C2. Gần một nửa nhãn của SemFault-250 không trỏ đúng dòng lỗi 44
7.1.3 C3. Một prompt có kiểm thử thất bại thắng quy trình nhiều bước 45
7.1.4 C4. Thêm phức tạp không thêm được gì — và vì sao 45

7.1.5 C5. Bảng kiểm tra đánh giá và gói tái hiện 45
7.2 Triết lý khoa học 46
7.2.1 Bằng chứng thực thi là trọng tài — và giới hạn của nó 46
7.2.2 Đo đóng góp, so với phương pháp đối chứng đơn giản, và chấp nhận bị bác bỏ 46
7.3 Các mối đe dọa đến tính hợp lệ 47
7.3.1 Tính hợp lệ nội tại và tính hợp lệ của cấu trúc 47
7.3.2 Tính hợp lệ ngoại tại 48

8 Kết luận và hướng phát triển 49
8.1 Kết luận 49
8.2 Hướng phát triển 50

A Hướng dẫn tái hiện và bảng bổ sung 51
A.1 Bốn mức tái hiện 51
A.1.1 Từ dữ liệu đến bảng 51
A.1.2 Môi trường 51
A.2 Bảng bổ sung 52
A.2.1 Phân rã trên nhật ký công bố 52
A.2.2 Kết quả đầy đủ trên ba bộ nhãn 53
A.3 Các prompt 54
A.3.1 LLM + kiểm thử thất bại (phương pháp đề xuất) và biến thể có độ phủ 54
A.3.2 Đề xuất bản sửa (bộ phán định nhãn và đề xuất rồi kiểm chứng) . 54
A.4 Quy trình duyệt nhãn 55

Danh sách bảng

2.1 Bộ từ khóa của đề tài 6
2.2 So sánh ba bài báo được giao với hướng của nhóm 12
4.1 Ba bộ dòng lỗi chuẩn dùng trong tiểu luận 22
5.1 Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình) 28
5.2 Nhãn gốc so với nhãn đã rà soát (cả 250 chương trình) 29
5.3 Kết quả chính trên ba bộ nhãn (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy) và chi phí token mỗi chương trình 30
5.4 Các cách bổ sung: chênh lệch so với phương pháp được mở rộng (ba dòng đầu trên tập test, 3 lần chạy, có khoảng tin cậy; hai dòng sau trên tập dev) 31
5.5 Phân tích tĩnh trên 250 chương trình (bộ nhãn đã rà soát) 32
5.6 Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần) . 33
5.7 RQ3a: 31 lỗi thật youtube-dl (BugsInPy), 3 lần chạy 35
5.8 RQ3b: bản gốc và bản đổi tên định danh (dòng lỗi chuẩn đã sửa, 3 cặp lần chạy) 36
5.9 RQ3c: năm mô hình trên cùng tập con (dòng lỗi chuẩn đã sửa, 3 lần chạy) 36
5.10 Trả lời ba câu hỏi nghiên cứu 37
6.1 Phân nhóm kết quả theo chương trình (dòng lỗi chuẩn đã sửa) 38
A.1 Phân rã Shapley trên nhật ký công bố của tác giả (dòng lỗi chuẩn gốc) . . 52
A.2 RQ2 đầy đủ: mọi biến thể trên ba bộ nhãn (tập test, Gemini 3.8 Flash, 3 lần chạy; Δ so với SemLoc) 53

Danh sách hình vẽ

3.1 Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn 16

5.1 Chi phí và độ chính xác (Acc@3) trên cùng tập con, dòng lỗi chuẩn đã sửa; đường nối hai phương pháp của cùng một mô hình 34

Danh mục từ viết tắt và thuật ngữ

Viết tắt / thuật ngữ	Ý nghĩa
Bộ phán định	Quy trình tự động quyết định một dòng có phải dòng lỗi hay không: một bản sửa nhỏ trên dòng đó làm mọi kiểm thử đạt (oracle)
Bootstrap	Lấy mẫu lại có hoàn lại để ước lượng khoảng tin cậy; ở đây lấy mẫu theo chương trình, 10000 lần
Grounded / guessed	Phần thứ hạng do bằng chứng thực thi quyết định / phần do LLM tự đoán
Bản sửa hợp lệ bề ngoài	Bản sửa làm mọi kiểm thử đạt nhưng không sửa đúng chỗ lỗi (plausible patch)
CI	Tích hợp liên tục: hệ thống tự chạy kiểm thử khi mã được đẩy lên kho
Phân tích tĩnh / động	Phân tích mã không chạy chương trình / dựa trên việc chạy chương trình
TDD	Phát triển hướng kiểm thử: viết kiểm thử trước khi viết mã
Phương pháp đối chứng	Phương pháp đơn giản dùng để so sánh (baseline)
Prompt (lời nhắc)	Văn bản đầu vào gửi cho LLM
Quy trình nhiều bước	Chuỗi các bước LLM và thực thi nối tiếp nhau (pipeline), như SemLoc
Token	Đơn vị văn bản mà LLM xử lý và tính phí
Acc@k	Tỉ lệ chương trình có dòng lỗi nằm trong k dòng bị nghi ngờ nhất (độ chính xác top-k)
CF	Kiểm chứng phản thực tế (counterfactual verification): vá thử một dòng rồi chạy lại kiểm thử
CI	Khoảng tin cậy (confidence interval), ở đây là khoảng 95% theo bootstrap
CovGate	Cách bổ sung do nhóm thử: kết hợp độ phủ với điểm của SemLoc
Đề xuất rồi kiểm chứng	Cách bổ sung do nhóm thử: LLM đề xuất dòng nghi ngờ, bản sửa nhỏ và kiểm thử xác nhận
DD	Gỡ lỗi chia đôi (delta debugging)
GT	Dòng lỗi chuẩn (ground truth) của bộ dữ liệu

Viết tắt / thuật ngữ	Ý nghĩa
LLM	Mô hình ngôn ngữ lớn (large language model)
RQ	Câu hỏi nghiên cứu (research question)
SBFL	Định vị lỗi dựa trên phổ độ phủ (spectrum-based fault localization)
SemFault-250	Bộ dữ liệu 250 chương trình Python có lỗi ngữ nghĩa của SemLoc
SemFault-250 đã rà soát	Bộ dữ liệu trên với nhãn được LLM rà trước và thành viên nhóm duyệt
SemFault-250 đã sửa	Bộ dữ liệu trên với dòng lỗi chuẩn được nhóm kiểm chứng và sửa lại bằng thực thi (tiếng Anh: SemFault-250-repaired)
Shapley	Giá trị Shapley: cách chia một tổng cho các thành phần theo đóng góp biên trung bình

Chương 1. Mở đầu

1.1 ĐẶT VẤN ĐỀ

Định vị lỗi là bước tốn công nhất của gỡ lỗi: khi chạy kiểm thử thấy thất bại — trên máy cá nhân hay trên hệ thống tích hợp liên tục (CI) — lập trình viên phải tìm ra dòng mã gây lỗi trong hàng trăm dòng có liên quan. Các công cụ phân tích tĩnh thông dụng như trình kiểm tra phong cách mã (lint) hay trình kiểm tra kiểu không giúp được ở bước này khi lỗi nằm ở logic: mã vẫn đúng cú pháp, đúng kiểu, chỉ cho ra kết quả sai. Thông tin duy nhất chỉ ra lỗi là chính các kiểm thử thất bại — và với cách phát triển hướng kiểm thử (TDD) [4], các kiểm thử đó được viết ra trước, mô tả ý định của mã.

Trợ lý AI đã trở thành công cụ hằng ngày của lập trình viên: các khảo sát lớn năm 2025 cho thấy phần lớn lập trình viên đang dùng hoặc định dùng chúng [5, 6], và gỡ lỗi là một trong các việc họ thường đem hỏi [7]. Nghiên cứu cũng cho thấy đưa thông tin lỗi cho LLM giúp nó định vị lỗi tốt hơn hẳn [8, 9]. Cách làm thực tế vì vậy rất đơn giản: chạy kiểm thử, đưa kết quả thất bại cho LLM, hỏi dòng nào sai.

Họ phương pháp kinh điển là định vị lỗi dựa trên phổ độ phủ (SBFL) [10, 11, 12]: một dòng càng hay được các kiểm thử thất bại chạy qua và càng ít được các kiểm thử thành công chạy qua thì càng đáng nghi. Với *lỗi ngữ nghĩa* — chương trình chạy đúng đường nhưng tính sai giá trị — hai loại kiểm thử thường chạy qua cùng một tập dòng, và SBFL gần như mất tín hiệu.

SemLoc [1] — bài báo trung tâm trong ba bài được giảng viên giao — đề xuất một quy trình nhiều bước cho lỗi ngữ nghĩa: LLM viết ra các ràng buộc ngữ nghĩa thực thi được, ràng buộc được chèn vào chương trình, kiểm thử được chạy để lấy phổ vi phạm, rồi LLM vá thử các dòng nghi ngờ để kiểm chứng. Bài báo tuyên bố độ chính xác đến từ việc *neo lập luận của LLM vào bằng chứng thực thi* (structured grounding) và báo cáo Acc@1 42,8% trên bộ dữ liệu SemFault-250. Câu hỏi tự nhiên với người làm phần

mềm là: **quy trình phức tạp đó có thật sự cần không, khi so với cách đơn giản mà lập trình viên vẫn làm?**

1.2 CÂU HỎI NGHIÊN CỨU

Tiểu luận đặt câu hỏi "grounded hay guessed?" cho SemLoc — độ chính xác đến từ bằng chứng thực thi hay từ việc LLM đoán — và tách thành ba câu hỏi nghiên cứu:

- **RQ1 – Tái hiện và kiểm tra cách đánh giá.** Kết quả công bố của SemLoc có tái hiện được không, và còn đứng vững khi chấm điểm công bằng, khi bỏ các thành phần không được mô tả trong bài báo, và khi kiểm tra lại chính các dòng lỗi chuẩn (nhãn) của bộ dữ liệu?
- **RQ2 – LLM thật sự cần gì?** Một prompt duy nhất có kiểm thử thất bại so với quy trình nhiều bước của SemLoc thì sao, về độ chính xác và chi phí? Thông tin nào trong prompt tạo ra khác biệt (kiểm thử thất bại hay độ phủ từng dòng)? Các cách bổ sung — gắn độ phủ vào SemLoc (CovGate), kiểm chứng dòng nghi ngờ bằng bản sửa và kiểm thử, biểu quyết nhiều câu trả lời — và các công cụ phân tích tĩnh có thêm được gì không?
- **RQ3 – Độ bền.** Các kết luận có giữ được trên lỗi thật (BugsInPy [13]), trên mã đã đổi tên định danh, trên năm họ mô hình, và trên cả ba bộ nhãn (gốc, đã sửa, đã rà soát) không?

1.3 ĐÓNG GÓP CHÍNH

Mỗi đóng góp dưới đây gắn với số liệu được sinh tự động từ mã nguồn công khai.

1. **Hạng 1 của SemLoc là dòng LLM đoán.** Mã nguồn công bố cộng cho dòng LLM tự đoán một điểm thưởng lớn hơn mọi điểm ràng buộc — chi tiết không có trong bài báo — nên dòng đầu tiên của SemLoc *luôn* là dòng đoán. Bỏ điểm thưởng,

Acc@1 đổi −9,1 điểm trên bộ nhãn đã sửa và −26,2 điểm trên bộ nhãn đã rà soát. Bước vá thử chiếm 42% lượt gọi LLM nhưng không đổi thứ hạng nào.

2. **Gần một nửa nhãn của bộ dữ liệu không trỏ đúng dòng lỗi.** Trong 250 nhãn gốc, 112 bị dời khi rà soát (56 nằm trên dòng trống, 32 lệch một dòng). Nhóm công bố hai bộ nhãn mới: *đã sửa* (chỉ kiểm thử quyết định) và *đã rà soát* (LLM rà trước, thành viên nhóm duyệt). Nhãn sai vô tình có lợi cho SemLoc: trên nhãn đã rà soát, phần hơn của SemLoc so với LLM hỏi thẳng ở Acc@3 biến mất.
3. **Một prompt có kiểm thử thất bại thắng quy trình nhiều bước.** Phương pháp do nhóm đề xuất — một lượt gọi LLM với mã nguồn, tên kiểm thử thất bại và thông báo lỗi — hơn SemLoc +10,2 điểm Acc@1 và +14,5 điểm Acc@3 trên nhãn đã sửa, thắng ở Acc@1 trên cả ba bộ nhãn, và tốn 2,0 nghìn token mỗi chương trình so với 14,0 nghìn của SemLoc.
4. **Thêm phức tạp không thêm được gì — và vì sao.** Ablation cho thấy kiểm thử thất bại là tín hiệu chính (Acc@3 +27,2 điểm so với LLM hỏi thẳng), còn thêm độ phủ từng dòng làm Acc@1 đổi −5,4 điểm. CovGate, kiểm chứng dòng nghi ngờ bằng bản sửa và biểu quyết đều không cải thiện được prompt đơn giản; công cụ phân tích tĩnh (ruff, pyrefly) gần như không trúng dòng lỗi. Nguyên nhân chung: kiểm thử của bộ dữ liệu đủ để *gợi ý* lỗi nhưng quá yếu để *làm trọng tài* — bản sửa nhỏ làm mọi kiểm thử đạt ở 23,3% lần thử trên dòng không có lỗi.
5. **Bảng kiểm tra bảy mục** cho đánh giá định vị lỗi bằng LLM, áp cho mọi phương pháp trong bài, và một gói tái hiện (Docker, nhật ký và bộ đệm lượt gọi LLM, mọi con số sinh tự động).

Chương 6 phân tích các ca thất bại và cơ chế khiến kiểm thử không làm được trọng tài; Chương 7 trình bày quan điểm khoa học cùng các mối đe dọa đến tính hợp lệ của kết quả.

1.4 CẤU TRÚC BÁO CÁO

Chương 2 tổng quan tài liệu và bộ từ khóa, trong đó phân tích ba bài báo được giao. Chương 3 trình bày cơ sở lý thuyết về SBFL, lỗi ngữ nghĩa, SemLoc, phân tích tĩnh và động, và các công cụ thống kê. Chương 4 mô tả câu hỏi nghiên cứu, dữ liệu và ba bộ nhãn, các phương pháp so sánh và thiết kế thực nghiệm. Chương 5 trả lời ba câu hỏi nghiên cứu. Chương 6 phân tích các ca thất bại; Chương 7 tổng hợp đóng góp, triết lý khoa học và các mối đe dọa. Chương 8 kết luận và nêu hướng phát triển. Phụ lục hướng dẫn tái hiện toàn bộ kết quả và liệt kê các prompt. Mã nguồn, dữ liệu và nhật ký gọi LLM được công bố tại `https://github.com/doanan293/grounded-or-guessed`.

Chương 2. Tổng quan tài liệu và từ khóa

2.1 PHƯƠNG PHÁP TÌM VÀ CHỌN TÀI LIỆU

2.1.1 Nguồn và tiêu chí chọn

Điểm xuất phát là ba bài báo giảng viên giao cho chương 10 “Phát hiện lỗi và định vị lỗi”: SemLoc [1], EffiHolmes [2] và báo cáo công nghiệp về khai thác báo cáo sự cố kết hợp LLM [3]. Từ ba bài này, nhóm mở rộng theo hai chiều: lần ngược các tài liệu được trích dẫn (định vị lỗi dựa trên phổ, dựa trên đột biến, gỡ lỗi chia đôi, các bộ dữ liệu lỗi) và tìm xuôi các nghiên cứu mới về định vị lỗi bằng LLM và về tính hợp lệ khi đánh giá LLM. Nguồn tra cứu là arXiv, ACM Digital Library, IEEE Xplore và Google Scholar; mỗi tài liệu được đối chiếu lại qua DOI hoặc mã arXiv trước khi đưa vào danh mục tham khảo.

Một tài liệu được chọn khi thỏa ít nhất một tiêu chí: (i) là phương pháp mà SemLoc so sánh hoặc kế thừa; (ii) là bộ dữ liệu lỗi dùng được cho Python; (iii) bàn về cách đánh giá định vị lỗi (hòa điểm, lỗi thiếu mã, lỗi nhân tạo so với lỗi thật); (iv) bàn về rủi ro khi dùng LLM trong nghiên cứu phần mềm (nhiễm dữ liệu, tái hiện); (v) cho biết lập trình viên thực sự dùng LLM để gỡ lỗi ra sao, hoặc bàn về nhãn sai và bản sửa “lách” kiểm thử. Kết quả là ba bài được giao và ba mươi mốt tài liệu bổ sung, tổng hợp ở Bảng 2.2.

2.1.2 Bộ từ khóa

Bảng 2.1 là bộ từ khóa song ngữ dùng để tìm kiếm và thống nhất thuật ngữ trong báo cáo.

Bảng 2.1: Bộ từ khóa của đề tài

Tiếng Anh	Tiếng Việt
fault localization	định vị lỗi
spectrum-based fault localization (SBFL)	định vị lỗi dựa trên phổ độ phủ
semantic fault	lỗi ngữ nghĩa
semantic constraint / invariant	ràng buộc ngữ nghĩa / bất biến
semantic violation spectrum	phổ vi phạm ràng buộc ngữ nghĩa
counterfactual verification	kiểm chứng phản thực tế
large language model (LLM)	mô hình ngôn ngữ lớn
structured grounding	neo lập luận vào biểu diễn có cấu trúc
program instrumentation	chèn mã kiểm tra vào chương trình
reproducibility	tính tái hiện được
data contamination / memorization	nhiễm dữ liệu huấn luyện / ghi nhớ
omission fault	lỗi thiếu mã
test-driven development (TDD)	phát triển hướng kiểm thử
plausible patch / test overfitting	bản sửa hợp lệ bề ngoài / quá khớp kiểm thử
label error (benchmark)	nhãn sai trong bộ dữ liệu
static / dynamic analysis	phân tích tĩnh / phân tích động

2.2 BA BÀI BÁO ĐƯỢC GIAO

2.2.1 SemLoc: neo lập luận của LLM vào ràng buộc thực thi được

Vấn đề. Với lỗi ngữ nghĩa, kiểm thử thành công và thất bại chạy qua cùng các câu lệnh, nên phổ độ phủ "sụp" và SBFL mất tín hiệu; còn LLM tự do thì ngẫu nhiên, không kiểm chứng được và khó tách nguyên nhân gốc với triệu chứng.

Phương pháp. SemLoc [1] gồm bốn bước: (1) tìm các *điểm neo* có cấu trúc trong chương trình (đầu hàm, điểm định nghĩa – sử dụng, biên vòng lặp, lệnh trả về); (2) một

lượt gọi LLM sinh các ràng buộc ngữ nghĩa theo một ngôn ngữ trung gian khép kín, mỗi ràng buộc gắn với một điểm neo và một biểu thức kiểm tra; (3) chèn các ràng buộc vào chương trình, chạy bộ kiểm thử, dựng ma trận "ràng buộc × kiểm thử" và chấm từng ràng buộc bằng công thức Ochiai, rồi truyền điểm về câu lệnh thiết lập tính chất đó; (4) kiểm chứng phản thực tế: với mỗi ràng buộc còn lại, LLM viết một bản vá một dòng tại dòng được quy cho ràng buộc, chạy lại kiểm thử và phân loại ràng buộc theo kết quả.

Đánh giá. Bộ dữ liệu SemFault-250 gồm 250 chương trình Python một hàm, một lỗi, mỗi chương trình có khoảng mười kiểm thử. SemLoc được so với SBFL (Ochiai, Tarantula), gỡ lỗi chia đôi và LLM hỏi thẳng, dùng Claude Sonnet 4.6 và Gemini 2.5 Pro ở ba mức nhiệt độ. Kết quả công bố: Acc@1 42,8%, Acc@3 68,0%; trên lỗi thật, bài báo chỉ có bảng cấp dòng cho bảy lỗi youtube-dl và tự nhận đây là kết quả khả thi.

Điểm cần kiểm chứng. Khi đọc mã nguồn công khai, nhóm thấy: bằng chứng trên lỗi thật rất ít; chương trình của SemFault-250 có văn phong đồng nhất như mã do LLM viết, nên khả năng LLM đã "thấy" dữ liệu chưa được kiểm tra; SemLoc bỏ hẳn tín hiệu độ phủ; mã chấm điểm phá hòa theo thứ tự dòng, tính mọi lần trúng của gỡ lỗi chia đôi là hạng 1 và loại các chương trình chạy lỗi khỏi mẫu số; mỗi cấu hình chỉ chạy một lần, không có kiểm định thống kê. Đây là đầu vào trực tiếp cho ba câu hỏi nghiên cứu.

2.2.2 EffiHolmes: định vị chỗ sửa lỗi hiệu năng bằng so sánh hai lần chạy

EffiHolmes [2] giải bài toán khó hơn ở một khía cạnh: lỗi hiệu năng không làm kiểm thử thất bại và không có vết ngăn xếp, nên các kỹ thuật định vị lỗi kinh điển không áp dụng được. Phương pháp chạy chương trình dưới hai mức tải (mặc định và phóng to), coi các hàm có chi phí tăng theo tải là điểm nóng, trích đường gọi gọn từ điểm vào tới điểm nóng, rồi để LLM lập luận có định hướng trên đường gọi đó. Trên 140 vấn đề hiệu năng thật từ năm kho mã khoa học dữ liệu, EffiHolmes vượt các phương pháp dựa trên truy hồi, tác tử và phân tích hiệu năng đơn thuần, và bền khi dùng mô hình nhỏ.

Bài học cho đề tài: cùng một ý tưởng với SemLoc — *đối chiếu hai lần thực thi để*

lọc nhiễu trước khi hỏi LLM — nhưng tín hiệu đối chiếu là chi phí theo tải thay vì kiểm thử đạt/không đạt. Cấu trúc câu hỏi nghiên cứu của bài (hiệu quả, loại bỏ thành phần, độ bền theo cỡ mô hình, phân tích thất bại) được nhóm dùng làm khuôn cho ba câu hỏi của mình. Lỗi hiệu năng nằm ngoài phạm vi thực nghiệm.

2.2.3 Khai thác báo cáo sự cố kết hợp LLM trong công nghiệp

Medeiros và cộng sự [3] khảo sát hai hệ thống Java lớn trong công nghiệp nhận hàng nghìn báo cáo sự cố mỗi tuần. Họ gom cụm vết ngăn xếp, xếp hạng tệp và phương thức đáng nghi, rồi đưa các xếp hạng này vào prompt của LLM; đánh giá hồi cứu năm LLM với bốn cấu hình prompt trên 38 lỗi sự cố thật, có kiểm tra thủ công. Cấu hình tốt nhất định vị được tới 71 trên 100 lỗi và sửa đúng 52 trên 100; thêm bảng xếp hạng phương thức làm tăng tỉ lệ câu trả lời nhắc tới phương thức đã được sửa.

Bài học cho đề tài: (i) *đưa bằng chứng chương trình đã khai thác (xếp hạng) vào prompt* giúp LLM định vị tốt hơn — đây chính là ý tưởng của phương pháp đối chứng LLM + độ phủ mà nhóm xây dựng; (ii) đầu ra của LLM cần được kiểm tra vì hay bị cắt cụt, lệch định dạng hoặc khẳng định không kiểm chứng. Bài cần dữ liệu công nghiệp nên không được dùng để thực nghiệm.

2.3 CÁC HƯỚNG NGHIÊN CỨU LIÊN QUAN

2.3.1 Định vị lỗi dựa trên phổ và đột biến

SBFL bắt đầu với Tarantula [10]; Ochiai [11] và DStar [14] là các công thức mạnh hơn được dùng làm mốc so sánh trong hầu hết nghiên cứu, và là nền của phổ vi phạm trong SemLoc. Các khảo sát của Wong và cộng sự [12] và de Souza và cộng sự [15] chỉ ra hai điểm yếu kinh điển của đánh giá SBFL: xử lý hòa điểm và lỗi thiếu mã (không có dòng nào để đổ lỗi). Định vị lỗi dựa trên đột biến (Metallaxis [16], MUSE [17]) thay độ phủ

bằng hành vi của các đột biến; gỡ lỗi chia đôi [18] và chuỗi nhân quả [19] là nền khái niệm cho bước kiểm chứng phản thực tế của SemLoc. Daikon [20] suy ra bất biến từ vết thực thi — SemLoc thay vào đó để LLM đề xuất bất biến.

2.3.2 Định vị lỗi bằng mô hình ngôn ngữ lớn

AutoFL [9] cho LLM gọi công cụ để duyệt mã và giải thích, ở cấp phương thức; SemLoc dùng AutoFL để chọn hàm trước khi xếp hạng dòng trên BugsInPy. AgentFL [21] mở rộng lên cấp dự án bằng nhiều tác tử. FlexFL [22] gần với ý tưởng của nhóm nhất: đưa kết quả SBFL làm gợi ý cho LLM mã nguồn mở, nhưng ở cấp phương thức trên mã Java. Điểm chung là bằng chứng thực thi được đưa vào dưới dạng văn bản; chưa nghiên cứu nào đo riêng phần độ chính xác đến từ bằng chứng và phần đến từ việc LLM tự đoán. Wu và cộng sự [8] cho thấy ChatGPT định vị lỗi tốt hơn hẳn khi được đưa nhật ký lỗi, tốt ở phạm vi một hàm nhưng giảm mạnh khi phải xét cả lớp. Ở bài toán sửa lỗi cấp kho mã, Agentless [23] cho thấy một quy trình hai bước đơn giản (định vị rồi sửa, không có tác tử tự chọn hành động) đạt kết quả tốt nhất trong các phương pháp mã nguồn mở trên SWE-bench Lite với chi phí thấp hơn nhiều — một tiền lệ cho câu hỏi "phức tạp có cần không".

2.3.3 Thực tế phát triển phần mềm

Các khảo sát lớn năm 2025 của Stack Overflow [5] và JetBrains [6] cho thấy phần lớn lập trình viên dùng hoặc định dùng trợ lý AI trong công việc, dù gần một nửa không tin độ chính xác của chúng. Bộ dữ liệu DevGPT [7] — các cuộc hội thoại thật giữa lập trình viên và ChatGPT được chia sẻ trên GitHub — cho thấy gỡ lỗi là một trong các nhóm việc thường được đem hỏi. Từ lâu, Parnin và Orso [24] đã chỉ ra rằng nhiều giả định của các kỹ thuật định vị lỗi tự động không khớp với cách lập trình viên thực sự gỡ lỗi, và khảo sát 386 người làm nghề của Kochhar và cộng sự [25] cho thấy người dùng có kỳ vọng cụ thể mà nghiên cứu chưa đáp ứng. Trong cách phát triển hướng kiểm thử [4], kiểm thử

được viết trước và mô tả ý định của mã — chính là thông tin mà một LLM cần để biết mã "sai" so với điều gì. Nhóm lấy cách làm phổ biến này — chạy kiểm thử, đưa kết quả thất bại cho LLM — làm mốc so sánh bắt buộc.

2.3.4 Bộ dữ liệu lỗi và tính hợp lệ của đánh giá

Defects4J [26] (Java) và BugsInPy [13] (Python) cung cấp lỗi thật kèm kiểm thử làm lộ lỗi; QuixBugs [27] gồm các chương trình nhỏ có lỗi một dòng. Pearson và cộng sự [28] cho thấy kết quả định vị lỗi trên lỗi nhân tạo không chuyển được sang lỗi thật, và cách chấm khi hòa điểm hay khi lỗi là thiếu mã có thể đảo thứ hạng giữa các kỹ thuật — đây là cơ sở cho phần kiểm tra cách đánh giá ở RQ1. Nhãn sai không phải chuyện hiếm: Northcutt và cộng sự [29] tìm thấy trung bình khoảng ba phần trăm nhãn sai trong tập kiểm tra của mười bộ dữ liệu học máy phổ biến, đủ để đảo thứ hạng giữa các mô hình. Agentless cũng phát hiện một phần đề bài của SWE-bench Lite có vấn đề và loại chúng ra khi đánh giá [23].

2.3.5 Bản sửa vượt kiểm thử nhưng không đúng

Trong sửa lỗi tự động, một bản vá làm mọi kiểm thử đạt được gọi là *hợp lệ bề ngoài* (plausible) — chưa chắc đã đúng. Qi và cộng sự [30] cho thấy phần lớn bản vá được báo cáo của các hệ thống sinh-và-kiểm-chứng (GenProg, RSRepair, AE) không đúng, nhiều bản chỉ tương đương với việc xóa bớt chức năng; Smith và cộng sự [31] gọi hiện tượng này là "quá khớp" bộ kiểm thử. Cùng một cơ chế chi phối mọi cách dùng kiểm thử làm trọng tài cho *vị trí* lỗi: bước vá thử của SemLoc, bộ kiểm chứng nhãn của nhóm và phương pháp "đề xuất rồi kiểm chứng" (Chương 5).

2.3.6 Nhiễm dữ liệu và tái hiện khi dùng LLM

Sallou và cộng sự [32] liệt kê các mối đe dọa khi dùng LLM trong nghiên cứu kỹ nghệ phần mềm: mô hình đóng thay đổi theo thời gian, nhiễm dữ liệu, khó tái hiện. Ramos và cộng sự [33] cho thấy LLM có dấu hiệu ghi nhớ các bộ dữ liệu lỗi phổ biến; khảo sát của Xu và cộng sự [34] đề xuất viết lại hoặc đổi tên để giảm nhiễm. Nhóm áp dụng trực tiếp: biến thể đổi tên định danh của SemFault-250 (RQ3), ghi nhật ký và lưu đệm mọi lượt gọi LLM để chạy lại được.

2.4 SO SÁNH VÀ KHOẢNG TRỐNG NGHIÊN CỨU

2.4.1 Bảng so sánh

Bảng 2.2 đặt ba bài được giao cạnh hướng tiếp cận của nhóm.

Bảng 2.2: So sánh ba bài báo được giao với hướng của nhóm

Tiêu chí	SemLoc [1]	EffiHolmes [2]	Báo cáo sự cố + LLM [3]	Nhóm M9
Loại lỗi	ngữ nghĩa	hiệu năng	sự cố (Java)	ngữ nghĩa
Tín hiệu	ràng buộc do LLM sinh + chạy kiểm thử	phân tích hiệu năng ở hai mức tải	vết ngăn xếp + xếp hạng	kiểm thử thất bại đưa thẳng cho LLM; tách đóng góp của từng tín hiệu
Lỗi thật	7 lỗi có kết quả cấp dòng	140 vấn đề	38 lỗi công nghiệp	31 lỗi BugsInPy, 3 lần chạy
Kiểm tra bộ dữ liệu	không	—	—	kiểm chứng bằng thực thi và rà soát toàn bộ nhãn
Lặp và kiểm định	1 lần, không kiểm định	—	—	3 lần, McNemar, bootstrap theo cụm

2.4.2 Khoảng trống và hướng của nhóm

Ba khoảng trống định hướng toàn bộ tiểu luận. Thứ nhất, các hệ thống định vị lỗi bằng LLM tuyên bố "neo vào thực thi" nhưng chưa ai đo phần độ chính xác thật sự đến từ bằng chứng thực thi. Thứ hai, kết quả được báo cáo từ một lần chạy với cách chấm lạc quan trên một bộ dữ liệu có nhãn chưa được kiểm tra. Thứ ba, chưa có so sánh với cách làm đơn giản mà lập trình viên vẫn dùng: một prompt duy nhất có kiểm thử thất bại. Từ đó, nhóm học và áp dụng: từ SemLoc — giả thuyết do LLM sinh nhưng kiểm thử phán xử, và dùng chính mã nguồn gốc để tái hiện; từ EffiHolmes — đối chiếu hai lần thực thi để lọc nhiễu (ý tưởng của CovGate) và khuôn câu hỏi nghiên cứu; từ báo cáo công nghiệp — đưa thông tin có sẵn vào prompt thay vì xây quy trình phức tạp; từ Pearson và

cộng sự, Northcutt và cộng sự — kiểm tra cách chấm và kiểm tra chính bộ nhãn; từ Qi và cộng sự — không tin một bản sửa chỉ vì nó làm kiểm thử đạt.

Chương 3. Cơ sở lý thuyết

3.1 ĐỊNH VỊ LỖI DỰA TRÊN PHỔ ĐỘ PHỦ

3.1.1 Ma trận độ phủ và các công thức nghi ngờ

Cho chương trình P và bộ kiểm thử $T = T_f \cup T_p$ gồm các kiểm thử thất bại T_f và thành công T_p. Với mỗi dòng s, ký hiệu $e_f(s)$, $e_p(s)$ là số kiểm thử thất bại, thành công chạy qua s; $n_f(s)$, $n_p(s)$ là số kiểm thử thất bại, thành công không chạy qua s. Ma trận độ phủ ghi lại, với mỗi cặp (kiểm thử, dòng), kiểm thử đó có chạy qua dòng đó hay không. SBFL gán cho mỗi dòng một điểm nghi ngờ; ba công thức dùng trong tiểu luận là Tarantula [10], Ochiai [11] và DStar [14]:

$$\text{Tarantula}(s) = \frac{e_f/(e_f + n_f)}{e_f/(e_f + n_f) + e_p/(e_p + n_p)}, \tag{3.1}$$

$$\text{Ochiai}(s) = \frac{e_f}{\sqrt{(e_f + n_f)\,(e_f + e_p)}}, \tag{3.2}$$

$$\text{DStar}(s) = \frac{e_f^*}{e_p + n_f}, \quad * = 2. \tag{3.3}$$

Các dòng được xếp giảm dần theo điểm; dòng không được kiểm thử thất bại nào chạy qua ($e_f = 0$) có điểm 0 và trong tiểu luận không được xếp hạng.

3.1.2 Chỉ số đánh giá và cách xử lý hòa điểm

Với mỗi chương trình, *hạng* của lỗi là hạng tốt nhất của một dòng lỗi chuẩn trong danh sách xếp hạng. Acc@k là tỉ lệ chương trình có hạng không vượt quá k:

$$\text{Acc@}k = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \mathbb{1}[\,\text{rank}_p \le k\,], \tag{3.4}$$

trong đó $\mathcal{P}$ là tập chương trình *cố định*: chương trình mà công cụ không chạy được vẫn nằm trong mẫu số và được tính là trượt. Khi nhiều dòng cùng điểm, có ba cách phá hòa: lấy vị trí đầu (lạc quan, theo thứ tự dòng), vị trí cuối (bi quan) hoặc *hạng trung bình* của nhóm hòa. Tiểu luận dùng hạng trung bình, theo khuyến nghị của các nghiên cứu về đánh giá định vị lỗi [28, 15]; cách chấm của tác giả SemLoc dùng vị trí đầu.

3.2 LỖI NGỮ NGHĨA

3.2.1 Vì sao độ phủ mất tín hiệu

Lỗi ngữ nghĩa là lỗi làm chương trình tính sai giá trị trong khi vẫn chạy qua cùng các câu lệnh như khi đúng — ví dụ `updated = cap - 1` thay vì `updated = cap`. Khi cả kiểm thử thành công và thất bại đều chạy qua dòng lỗi, $e_p(s) > 0$ làm điểm Ochiai của dòng lỗi giảm và ngang với các dòng xung quanh; SBFL khi đó gần như đoán mò trong phạm vi hàm.

3.2.2 Định nghĩa khách quan bằng độ phủ

Để phân tích kết quả theo loại lỗi, tiểu luận dùng một định nghĩa tự động, chỉ dựa trên độ phủ của các dòng lỗi chuẩn:

- **lỗi ngữ nghĩa** (semantic): có dòng lỗi được ít nhất một kiểm thử thất bại *và* một kiểm thử thành công chạy qua — độ phủ không phân biệt được;
- **lộ qua độ phủ** (coverage-revealed): kiểm thử thất bại chạy tới dòng lỗi nhưng không kiểm thử thành công nào chạy tới;
- **không thực thi** (not executed): không kiểm thử thất bại nào chạy tới dòng lỗi chuẩn — thường do dòng lỗi chuẩn bị đặt sai hoặc là dòng trống.

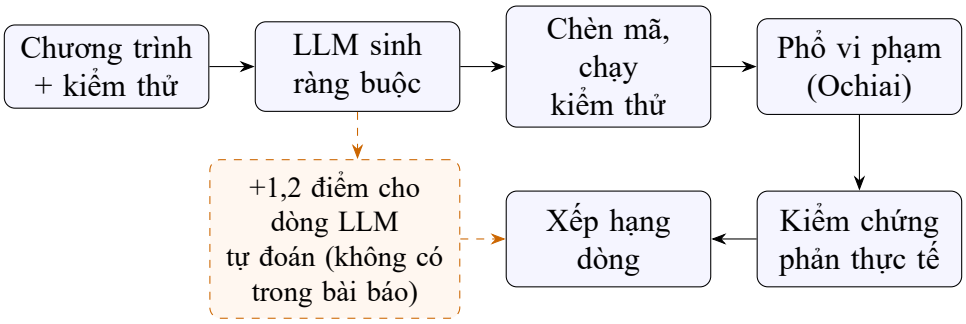
Nhãn được tính lại theo đúng bộ dòng lỗi chuẩn đang dùng để chấm (gốc hoặc đã kiểm

chứng).

3.3 SEMLOC: PHỔ VI PHẠM RÀNG BUỘC NGỮ NGHĨA

3.3.1 Quy trình bốn bước

Hình 3.1 tóm tắt quy trình của SemLoc [1]. Mỗi ràng buộc c do LLM sinh ra là một bộ $\langle$mã, loại, vùng, điểm neo, biểu thức, ý đồ$\rangle$; biểu thức được chèn vào chương trình dưới dạng một lời gọi kiểm tra tại điểm neo. Chạy bộ kiểm thử cho ma trận "ràng buộc $\times$ kiểm thử" ghi ràng buộc nào bị vi phạm trong kiểm thử nào; thay "dòng" bằng "ràng buộc" trong công thức (3.2) cho điểm của từng ràng buộc, và điểm của một dòng là điểm lớn nhất trong các ràng buộc được quy về dòng đó. Bước kiểm chứng phản thực tế loại ràng buộc bị vi phạm cả trong kiểm thử thành công, rồi cho LLM vá thử tại dòng được quy cho từng ràng buộc còn lại và chạy lại kiểm thử.



Hình 3.1: Quy trình của SemLoc (theo [1]); khung nét đứt là thành phần nhóm phát hiện trong mã nguồn

3.3.2 Điểm thưởng cho dòng LLM tự đoán

Khi đọc mã nguồn công khai, nhóm thấy một thành phần không được mô tả trong bài báo: ngoài các ràng buộc, LLM còn trả về một dòng lỗi mà nó tự đoán (trường `fault_line`), và dòng này được cộng thêm 1,2 điểm — lớn hơn điểm tối đa 1 của một ràng buộc. Vì vậy, dòng LLM tự đoán *luôn* đứng hạng 1, bất kể phổ vi phạm ra sao. Thành phần này

là trọng tâm của câu hỏi "grounded hay guessed" ở các chương sau.

3.4 PHÂN TÍCH TĨNH VÀ PHÂN TÍCH ĐỘNG

Phân tích tĩnh xét mã nguồn mà không chạy chương trình. Trình kiểm tra phong cách mã (lint) như ruff phát hiện mẫu mã dễ lỗi hoặc khó đọc (biến không dùng, so sánh thừa, cách viết dài dòng); trình kiểm tra kiểu như pyrefly suy kiểu và báo chỗ kiểu không khớp với chú thích. Cả hai chỉ biết mã *được viết* thế nào, không biết mã *được mong đợi* làm gì: viết > thay cho >= hay trừ phí hai lần đều hợp lệ về cú pháp và về kiểu. Phân tích động lấy thông tin từ việc chạy chương trình: kết quả kiểm thử, độ phủ, giá trị biến. Với lỗi ngữ nghĩa, chỉ kiểm thử — bản đặc tả do lập trình viên viết — mới chứa thông tin về ý định, nên định vị lỗi ngữ nghĩa thuộc về phân tích động; phân tích tĩnh được dùng như một mốc so sánh (Chương 5).

3.5 CÔNG CỤ THỐNG KÊ

3.5.1 Kiểm định cặp

Mọi so sánh giữa hai phương pháp đều là so sánh *cặp* trên cùng các chương trình. Với chỉ số nhị phân (trúng hay trượt top-k), kiểm định McNemar dùng số chương trình chỉ phương pháp A trúng (b) và chỉ phương pháp B trúng (c), bản chính xác khi $b + c$ nhỏ. Với hạng nghịch đảo ($1/\text{rank}$, trượt là 0), kiểm định dấu hạng Wilcoxon cùng hệ số tương quan hạng hai chuỗi ghép cặp làm độ lớn hiệu ứng:

$$r_{rb} = \frac{R^+ - R^-}{R^+ + R^-}, \tag{3.5}$$

với R^+, R^- là tổng hạng của các chênh lệch dương, âm (bỏ chênh lệch bằng 0).

3.5.2 Khoảng tin cậy bootstrap theo cụm và giá trị Shapley

Mỗi cấu hình được chạy ba lần. Kết quả của một chương trình được lấy trung bình qua ba lần *trước khi* lấy mẫu lại, nên đơn vị lấy mẫu là chương trình chứ không phải lần chạy. Vì nhiều chương trình của SemFault-250 sinh ra từ cùng một khuôn mẫu, khoảng tin cậy 95% được tính bằng bootstrap *theo cụm khuôn mẫu* (lấy lại cả cụm), với 10000 lần lấy mẫu và hạt giống cố định.

Để đo phần đóng góp của từng tín hiệu, tiểu luận dùng giá trị Shapley. Với tập tín hiệu N và hàm giá trị $v(S)$ là Acc@k khi chỉ bật các tín hiệu trong S,

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \Big(v(S \cup \{i\}) - v(S) \Big), \tag{3.6}$$

và $\sum_i \phi_i = v(N)$. Vì Acc@k là trung bình các giá trị trúng/trượt theo chương trình, giá trị Shapley tổng bằng trung bình giá trị Shapley của từng chương trình; tính chất này cho phép tính khoảng tin cậy bootstrap cho từng ϕ_i.

Chương 4. Phương pháp và thiết kế thực nghiệm

4.1 CÂU HỎI NGHIÊN CỨU VÀ GIẢ THUYẾT

4.1.1 Ba câu hỏi nghiên cứu

RQ1 (tái hiện và kiểm tra cách đánh giá): các con số công bố của SemLoc có tái hiện được từ dữ liệu và mã nguồn của tác giả không; chúng thay đổi thế nào khi (a) chấm điểm công bằng, (b) bỏ các thành phần không được mô tả trong bài báo, (c) kiểm tra lại chính các dòng lỗi chuẩn của bộ dữ liệu, (d) chạy lại với mô hình khác?

RQ2 (LLM thật sự cần gì?): trên cùng các chương trình và cùng mô hình, (a) một prompt duy nhất có kiểm thử thất bại so với quy trình nhiều bước SemLoc thế nào về độ chính xác và chi phí; (b) trong prompt đó, thông tin nào tạo ra khác biệt — kiểm thử thất bại hay độ phủ từng dòng; (c) các cách bổ sung (CovGate cho SemLoc; kiểm chứng dòng nghi ngờ bằng bản sửa; biểu quyết nhiều câu trả lời) và công cụ phân tích tĩnh có thêm được gì; (d) bao nhiêu phần độ chính xác của SemLoc đến từ bằng chứng thực thi, bao nhiêu từ dòng LLM tự đoán?

RQ3 (độ bền): các kết luận của RQ2 có còn đúng (a) trên lỗi thật của BugsInPy, (b) khi đổi tên mọi định danh và xóa chú thích của chương trình (kiểm tra ghi nhớ), (c) trên năm mô hình thuộc các họ khác nhau, (d) trên cả ba bộ nhãn?

4.1.2 Giả thuyết và tiêu chí bác bỏ

H1: *ở hạng 1, độ chính xác của SemLoc đến từ dòng LLM tự đoán; bằng chứng thực thi đóng góp ở top-3*. Bác bỏ nếu riêng phổ vi phạm đạt Acc@1 không thấp hơn SemLoc, hoặc nếu ở top-3 phổ không làm tăng độ chính xác so với riêng dòng đoán. **H2:** *một prompt có kiểm thử thất bại không kém quy trình nhiều bước*; bác bỏ nếu khoảng tin cậy của chênh lệch Acc@1 hoặc Acc@3 so với SemLoc nằm hẳn dưới 0. **H3:** *thêm độ phủ*

từng dòng vào prompt cải thiện kết quả; bác bỏ nếu khoảng tin cậy của chênh lệch không nằm hẳn trên 0. Mọi giả thuyết được kiểm trên cả ba bộ nhãn.

4.2 DỮ LIỆU

4.2.1 SemFault-250 và cách chia tập

SemFault-250 [1] gồm 250 chương trình Python một hàm, mỗi chương trình có đúng một lỗi được cài vào và khoảng mười kiểm thử. Nhóm chia cố định 50 chương trình làm tập phát triển (dev) — chỉ dùng để chọn tham số — và 200 chương trình làm tập kiểm tra (test) để báo cáo, phân tầng theo nhãn lỗi (mục 3.2) với hạt giống cố định. Vì nhiều chương trình được sinh từ cùng một khuôn mẫu, nhóm gom các chương trình có cấu trúc giống nhau thành cụm (độ tương đồng 4-gram Jaccard từ bốn phần năm trở lên, trên dạng chuẩn hóa của mã) để dùng trong bootstrap theo cụm.

4.2.2 SemFault-250 đã sửa: kiểm chứng dòng lỗi chuẩn bằng thực thi

Kiểm tra dòng lỗi chuẩn cho thấy 56 dòng là dòng trống và 65 chương trình không có kiểm thử thất bại nào chạy tới dòng lỗi chuẩn. Nhóm xây một *bộ phán định sửa tối thiểu*: một dòng được coi là vị trí lỗi nếu thay nó bằng một dòng khác (hoặc chèn một dòng vào dòng trống, với lỗi thiếu mã) với *thay đổi nhỏ* làm *mọi* kiểm thử đạt. LLM chỉ đề xuất các bản sửa ứng viên (GPT-6 luna, hai rồi bốn lần thử ở $T = 0,8$; lượt thứ hai với Claude Sonnet 5.5 cho các dòng còn chưa kiểm chứng); kiểm thử quyết định. Bản sửa được coi là nhỏ khi khoảng cách chỉnh sửa theo từ tố (định danh, số, từng ký tự dấu) so với dòng gốc tối đa là hai hoặc tối đa một phần ba số từ tố của dòng. Các dòng lân cận (trong phạm vi bốn dòng) chỉ được tìm khi không kiểm thử thất bại nào chạy tới dòng chuẩn gốc; vì vậy bộ phán định không phát hiện được trường hợp hai dòng cùng sửa được lỗi. Kết quả: 82 chương trình được *xác nhận* (sửa được ngay tại dòng chuẩn), 46 được *sửa vị trí* (dòng chuẩn không chạy tới được nhưng một dòng gần đó sửa được), 111

chưa kiểm chứng được (dòng chuẩn chạy tới được nhưng không bản sửa đề xuất nào đạt) và 11 *không kiểm chứng được* (bị loại). Vị trí "sửa" nằm trên dòng trống không phải câu lệnh nào để xếp hạng nên bị loại; bộ dòng lỗi chuẩn đã sửa còn 226 chương trình. Mọi kết quả được báo cáo trên cả bộ gốc và bộ đã sửa.

4.2.3 SemFault-250 đã rà soát: LLM rà trước, nhóm duyệt

Bộ phán định sửa tối thiểu chỉ có thể xác nhận một dòng khi tìm được bản sửa nhỏ, và chỉ tìm dòng lân cận khi dòng gốc không chạy tới được. Để kiểm tra *mọi* nhãn, nhóm làm thêm một vòng rà soát hai bước. (1) Một LLM (Claude Fable 5.1, mức suy luận cao) đọc toàn bộ chương trình, kiểm thử và nhãn gốc của cả 250 chương trình và cho một nhận định: nhãn đúng, nhãn sai (kèm dòng đề xuất), hoặc lỗi trải trên hai dòng (cả hai đều chấp nhận được); với nhiều ca, LLM tự chạy kiểm thử trên bản sửa đề xuất để kiểm tra. (2) 6 thành viên của nhóm duyệt lại 111 nhãn: 46 nhãn bộ phán định đã dời và mọi nhãn LLM cho là nằm sai dòng; mỗi ca có đoạn mã, liên kết tới chương trình và kiểm thử, các dòng ứng viên và nhận định của LLM để tham khảo. Người duyệt có xem gợi ý của LLM và đồng ý với gợi ý ở 111/111 ca. Trưởng nhóm sửa thêm 5 ca có bằng chứng chạy kiểm thử (hai ca lỗi hai dòng mà sửa một dòng không đủ, hai ca lỗi thiếu mã mà người duyệt chọn điểm chèn là dòng trống — dùng câu lệnh `return` như các chương trình cùng họ — và một ca LLM trích đúng câu lệnh nhưng ghi nhầm số dòng). Kết quả là bộ *đã rà soát*: đủ 250 chương trình, không nhãn nào trên dòng trống, 44 nhãn hai dòng.

Bảng 4.1 tóm tắt ba bộ nhãn. Bộ *đã sửa* chỉ do kiểm thử quyết định nên được dùng cho kết quả chính; bộ *đã rà soát* đầy đủ hơn nhưng nhãn do một LLM đề xuất, nên các phương pháp dùng LLM có thể được lợi (Chương 7) — nó được dùng như một góc nhìn độ nhạy.

Bảng 4.1: Ba bộ dòng lỗi chuẩn dùng trong tiểu luận

Bộ nhãn	Ai quyết định	Vai trò
Gốc	Tác giả SemLoc	So với bài báo gốc
Đã sửa	Chỉ kiểm thử (bộ phán định sửa tối thiểu)	Kết quả chính; không có ý kiến LLM
Đã rà soát	LLM rà trước, thành viên nhóm duyệt, trưởng nhóm sửa ca có bằng chứng chạy kiểm thử	Góc nhìn độ nhạy; đủ mọi chương trình

4.2.4 Lỗi thật BugsInPy

Nhóm dựng môi trường cho các lỗi của dự án youtube-dl trong BugsInPy [13] và giữ những lỗi thỏa đủ bốn điều kiện: môi trường cài được, kiểm thử của lỗi tái hiện được thất bại trên phiên bản lỗi, đạt trên phiên bản đã sửa, và đo được độ phủ — được 31 lỗi. Hàm đích là hàm chứa dòng bị lập trình viên sửa mà kiểm thử thất bại được khai báo thực sự chạy qua; dòng lỗi chuẩn là các dòng bị sửa trong hàm đó. Kiểm thử thất bại do môi trường (không được BugsInPy khai báo) bị loại khỏi ma trận độ phủ.

4.2.5 Biến thể đổi tên định danh

Để kiểm tra khả năng mô hình ghi nhớ SemFault-250 [33, 34], nhóm đổi tên mọi định danh do chương trình tự định nghĩa (hàm, tham số, biến, hằng) thành tên vô nghĩa, xóa chú thích và docstring nhưng *giữ nguyên số dòng*, chỉ giữ các tên mà kiểm thử phụ thuộc. Mỗi biến thể được kiểm tra tự động: cùng số dòng, cùng kết quả đạt/không đạt của từng kiểm thử như bản gốc; chương trình nào lệch bị loại. Kết quả là 199 chương trình của tập test.

4.3 CÁC PHƯƠNG PHÁP SO SÁNH

4.3.1 Các phương pháp có sẵn

- **SBFL Ochiai, DStar** trên ma trận độ phủ do nhóm thu thập (mỗi kiểm thử một ngữ cảnh độ phủ).
- **LLM hỏi thẳng**: chế độ dự đoán trực tiếp của SemLoc — một lượt gọi chỉ có mã nguồn chương trình, LLM trả về các dòng nghi ngờ; không có thông tin kiểm thử nào.
- **SemLoc** như phát hành, chạy qua mã nguồn gốc của tác giả, chỉ thay lớp gọi LLM bằng cổng của nhóm (cùng prompt, cùng logic).

4.3.2 LLM + kiểm thử thất bại: phương pháp đề xuất

Phương pháp chính của nhóm làm đúng điều lập trình viên vẫn làm: một lượt gọi LLM duy nhất (nhiệt độ $T = 0,8$) với mã nguồn đánh số dòng, số kiểm thử đạt/không đạt, câu "chương trình có đúng một dòng lỗi", và tên cùng dòng thông báo lỗi đầu tiên của mỗi kiểm thử thất bại; LLM trả về năm dòng nghi ngờ nhất dưới dạng JSON (nội dung prompt ở Phụ lục A). Prompt được thiết kế trước khi xem kết quả trên tập test và không được tinh chỉnh. Ý tưởng đưa thông tin sẵn có vào prompt thay vì xây quy trình nhiều bước lấy từ báo cáo công nghiệp [3] và các nghiên cứu cho thấy nhật ký lỗi giúp LLM định vị tốt hơn [8].

4.3.3 Ablation: tách độ phủ khỏi kiểm thử thất bại

Biến thể *LLM + kiểm thử thất bại + độ phủ* giữ nguyên prompt trên và chú thích thêm cho mỗi dòng số kiểm thử thất bại và thành công chạy qua nó (cùng một câu giải thích chú thích). Hai so sánh ghép cặp tách hai nguồn thông tin: (LLM + kiểm thử thất bại) so

với LLM hỏi thẳng đo tác dụng của kiểm thử thất bại; (LLM + kiểm thử thất bại + độ phủ) so với (LLM + kiểm thử thất bại) đo tác dụng của độ phủ.

4.3.4 Các cách bổ sung

CovGate lấy điểm dòng của SemLoc và bổ sung bằng chứng độ phủ qua bốn thành phần có thể bật/tắt: (1) *cổng độ phủ* — bỏ các dòng không kiểm thử thất bại nào chạy qua; (2) *cộng* λ·Ochiai vào điểm của dòng, với $\lambda \in \{0; 0,01; 0,1; 0,5\}$; (3) *xếp lại bằng kết quả kiểm chứng phản thực tế*; (4) *chỉ thưởng khi có căn cứ* — giữ điểm thưởng cho dòng LLM đoán chỉ khi dòng đó có bằng chứng thực thi. Tổ hợp tham số được chọn chỉ trên tập dev; cấu hình được chọn không bật cổng độ phủ, giữ nguyên điểm thưởng và dùng $\lambda = 0,01$ (bộ gốc) hoặc $\lambda = 0,5$ (bộ đã sửa) — tức là một bước *phá hòa bằng Ochiai* cho SemLoc.

Đề xuất rồi kiểm chứng dùng kiểm thử làm trọng tài cho vị trí lỗi: (1) LLM + kiểm thử thất bại đề xuất các dòng nghi ngờ; (2) với k dòng đầu, LLM viết một bản sửa nhỏ tại đúng dòng đó (cùng bộ đề xuất bản sửa và cùng tiêu chí "thay đổi nhỏ" như bộ phán định nhãn), chương trình đã vá được chạy với bộ kiểm thử; dòng có bản sửa làm mọi kiểm thử đạt được đưa lên đầu. Tham số $k \in \{2, 3\}$, số lần thử mỗi dòng (1 hoặc 2), thứ tự giữa các dòng được xác nhận (theo LLM hoặc theo kích thước bản sửa) và giới hạn kích thước bản sửa (1, 2, 3 từ tố hoặc không giới hạn) tạo thành một lưới 32 cấu hình. Mọi lần thử được ghi lại, nên cả lưới được đánh giá offline từ một lượt chạy với cấu hình rộng nhất (3 dòng, 2 lần thử) trên *tập dev*, 3 lần. Quy tắc chọn được cố định trong mã trước khi chạy: Acc@1 trung bình trên bộ nhãn gốc và bộ đã rà soát (không dùng bộ đã sửa vì bộ đó được dựng bằng chính cơ chế sửa thử).

Biểu quyết: gộp ba câu trả lời của LLM + kiểm thử thất bại bằng tổng nghịch đảo hạng, cũng trên tập dev.

4.3.5 Phân tích tĩnh

Ruff (luật mặc định và mọi luật) và pyrefly (kiểm tra kiểu), phiên bản cố định, được chạy trên cả 250 chương trình. Một chương trình được tính là "trúng" khi có cảnh báo nằm đúng dòng lỗi chuẩn. Để biết con số trúng có ý nghĩa không, nhóm tính số trúng kỳ vọng nếu các dòng có cảnh báo được rải ngẫu nhiên trên các dòng mã của chương trình.

4.4 KIỂM TRA CÁCH ĐÁNH GIÁ VÀ PHÂN RÃ ĐÓNG GÓP

4.4.1 Kiểm tra cách đánh giá của SemLoc

Từ nhật ký công bố của tác giả, nhóm tính lại mọi cấu hình theo hai cách: *cách chấm của tác giả* (phá hòa theo thứ tự dòng, gỡ lỗi chia đôi trúng bất kỳ dòng nào tính là hạng 1, chương trình chạy lỗi bị loại khỏi mẫu số) và *cách chấm công bằng* (hạng trung bình khi hòa, mẫu số cố định, công thức (3.4)). Nhóm cũng loại bỏ lần lượt các thành phần không được mô tả trong bài báo (điểm thưởng dòng đoán) và kiểm tra bước kiểm chứng phản thực tế có làm thay đổi thứ hạng hay không.

4.4.2 Phân rã "grounded hay guessed"

Điểm dòng của SemLoc là tổng của hai tín hiệu: phổ vi phạm ràng buộc (bằng chứng thực thi) và điểm thưởng cho dòng LLM tự đoán. Nhóm định nghĩa trò chơi hai người chơi {dòng đoán, phổ}: $v(S)$ là Acc@k khi chỉ giữ các tín hiệu trong S; khi giữ cả hai, điểm dòng trùng đúng với SemLoc như phát hành. Giá trị Shapley (3.6) chia Acc@k của SemLoc cho hai tín hiệu; một trò chơi mở rộng bốn người chơi thêm độ phủ (Ochiai) và xếp lại bằng kiểm chứng phản thực tế.

4.5 THIẾT LẬP THỰC NGHIỆM

4.5.1 Mô hình và hạ tầng

Mô hình chính là Gemini 3.8 Flash; RQ3 thêm GPT-6 luna, Claude Sonnet 5.5, gpt-oss-120b (mã nguồn mở) và Gemini 3.5 Flash-Lite trên một tập con cố định 50 chương trình của tập test; Claude Sonnet 4.6 chỉ dùng để đối chiếu với nhật ký của tác giả. Mọi mô hình chạy ở mức suy luận mặc định của nhà cung cấp, qua một cổng tương thích OpenAI; SemLoc chạy ở nhiệt độ $T = 0,8$ như cấu hình chính của bài báo. Mọi lượt gọi LLM được ghi nhật ký (mô hình thực sự trả lời, số token, thời gian, lỗi) và lưu đệm theo nội dung và mã lượt chạy, nên toàn bộ thực nghiệm phát lại được mà không gọi mạng.

4.5.2 Lặp, kiểm tra sức khỏe lượt chạy và thống kê

Mỗi cấu hình có LLM chạy 3 lần độc lập; mỗi lần là một mã lượt chạy và một thư mục làm việc riêng. Một lượt chạy chỉ được dùng khi đã chạy xong và qua kiểm tra sức khỏe: không còn lỗi gọi LLM chưa khắc phục, không có chương trình nào lỗi ở bước kiểm chứng phản thực tế. Kết quả báo cáo là trung bình $\pm$ độ lệch chuẩn qua 3 lần; so sánh cặp dùng chênh lệch Δ kèm khoảng tin cậy bootstrap theo cụm, McNemar cho từng lần chạy và Wilcoxon kèm r_{rb} (mục 3.5). Chi phí được đo bằng số lượt gọi, số token và thời gian trên mỗi chương trình.

Chương 5. Kết quả thực nghiệm

Mọi con số trong chương là phần trăm (Acc@k) hoặc điểm phần trăm (Δ), sinh tự động từ dữ liệu; khoảng trong ngoặc vuông là khoảng tin cậy 95% bootstrap theo cụm khuôn mẫu. Có khoảng ba mươi phép so sánh như vậy và nhóm không hiệu chỉnh đa so sánh, nên các khoảng sát 0 cần được đọc thận trọng. Trừ khi ghi khác, kết quả là của Gemini 3.8 Flash, ba lần chạy, trên tập test.

5.1 RQ1 – TÁI HIỆN VÀ KIỂM TRA CÁCH ĐÁNH GIÁ

5.1.1 Tái hiện và hai cách chấm điểm

Tính lại từ nhật ký công bố khớp từng con số của cả 13 cấu hình trong bảng kết quả của SemLoc. *Chạy lại* SemLoc với chính mô hình của bài báo (Claude Sonnet 4.6) trên tập dev thì không khớp: Acc@1 32,0% và Acc@3 44,0% so với 36,0% và 68,0% trong nhật ký trên cùng các chương trình; ràng buộc sinh ra ít bị vi phạm trong kiểm thử thất bại hơn, và nhóm không tìm thấy nguyên nhân ở hạ tầng của mình.

Bảng 5.1 so sánh cách chấm của tác giả với cách chấm công bằng. Ba điểm nổi bật:

- **Gỡ lỗi chia đôi**: mã chấm điểm công bố cho 38,0% (mọi lần trúng tính là hạng 1), trong khi bài báo ghi 0 và cách chấm công bằng cho 0,0%, vì phương pháp này không xếp hạng dòng nào.
- **SBFL bị đánh giá thấp ở top-5**: Acc@5 của Ochiai là 30,4% theo cách của tác giả và 48,0% khi dùng hạng trung bình.
- **SemLoc** gần như không đổi khi chấm công bằng, nhưng **bỏ điểm thưởng cho dòng LLM tự đoán** — thành phần không có trong bài báo — làm Acc@1 đi từ 42,8% xuống 30,0% (Claude) và từ 39,6% xuống 27,2% (Gemini).

Bảng 5.1: Kết quả công bố tính lại theo hai cách chấm (nhật ký của tác giả, 250 chương trình)

	Cách chấm của tác giả			Cách chấm công bằng		
Cấu hình (cả 250 chương trình)	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3	Acc@5
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	42,8	68,0	76,8	42,8	65,6	76,8
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	39,8	64,7	73,5	39,6	64,4	72,8
SemLoc Claude $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	30,0	55,6	66,4
SemLoc Gemini $T{=}0,8$ không có điểm thưởng dòng đoán	–	–	–	27,2	55,6	64,0
SemLoc không CF (lượt chạy riêng), Claude	30,8	59,6	68,8	–	–	–
LLM hỏi thẳng, Claude	35,2	41,2	41,2	35,2	41,2	41,2
SBFL Ochiai	6,4	13,2	30,4	4,0	16,0	48,0
SBFL DStar (tính lại)	–	–	–	4,0	16,0	48,0
Gỡ lỗi chia đôi (DD)	38,0	38,0	38,0	0,0	0,0	0,0

5.1.2 Kiểm chứng phản thực tế và lỗi của gói mã công bố

Trong mã nguồn công bố, kết quả của bước kiểm chứng phản thực tế được tính ra nhưng *không dùng để xếp hạng*: thứ hạng có và không có bước này trùng nhau ở cả 250 chương trình của cả năm cấu hình có dữ liệu, dù bước này chiếm 42% số lượt gọi LLM của SemLoc. Con số "không có kiểm chứng phản thực tế" trong bài báo đến từ một lượt chạy riêng không công bố điểm từng chương trình; Acc@1 của nó (30,8%) gần bằng SemLoc khi bỏ điểm thưởng, gợi ý một khác biệt về cấu hình hơn là tác dụng của bước này, nhưng nhóm không xác định được. Bộ chạy BugsInPy của SemLoc có thêm hai lỗi: (i) chỉ số "hạng tốt nhất" nó báo cáo là hạng của *chính dòng LLM đoán*, không phải của dòng lập trình viên đã sửa; (ii) bước vá thử sửa tệp nguồn rồi khôi phục trong cùng một giây với cùng kích thước tệp, để lại bản biên dịch Python (`.pyc`) của phiên bản *đã vá*, khiến lỗi "biến mất" ở lượt chạy sau. Nhóm đã sửa lỗi thứ hai trong bộ chạy của mình và chấm lại BugsInPy theo dòng đã sửa thật.

5.1.3 Chất lượng nhãn của bộ dữ liệu

Bảng 5.2 so nhãn gốc với nhãn đã rà soát trên cả 250 chương trình. Chỉ 98 nhãn giữ nguyên; 40 nhãn được giữ và thêm một dòng (lỗi trải hai dòng); 112 nhãn bị dời: nằm trên dòng trống (56), trên lệnh log, dòng `def` hoặc chú thích (7), lệch một dòng (32) hoặc xa hơn (17). Nhãn lệch thường đi theo khuôn mẫu: nhãn đặt ở dòng điều kiện trong khi lỗi nằm ở câu lệnh mà điều kiện bảo vệ (hoặc ngược lại), hay nhãn ở điểm chèn của một lỗi thiếu mã — tức một dòng trống. Vì bộ dữ liệu được sinh tự động, nhóm cho rằng đây là lỗi của công cụ ghi nhãn hơn là của người gán nhãn, nhưng không kiểm chứng được giả thuyết này khi không có mã sinh dữ liệu.

Bảng 5.2: Nhãn gốc so với nhãn đã rà soát (cả 250 chương trình)

Bản gốc label vs. reviewed label ($n = 250$)	Số chương trình
Cùng dòng	98
Giữ nguyên, thêm một dòng (lỗi trải hai dòng)	40
Dòng trống	56
Lệnh log, dòng `def` hoặc chú thích	7
Lệch một dòng	32
Lệch từ hai dòng trở lên	17
Bị dời (dòng trống, không phải mã, lệch một dòng trở lên)	112

Nhãn lệch làm mọi con số tuyệt đối thấp đi và, quan trọng hơn, *có lợi cho SemLoc*. Trên nhãn gốc, SemLoc hơn LLM hỏi thẳng 60,3% so với 43,0% ở Acc@3; trên nhãn đã rà soát, hai phương pháp ngang nhau (90,0% so với 90,8%; Δ của LLM hỏi thẳng +0,8 [−2,4; +4,0]) và LLM hỏi thẳng còn hơn ở Acc@1 (+4,3 [+0,3; +8,6]). Giải thích hợp lý: LLM hỏi thẳng thường chỉ nêu một hai dòng (Acc@3 gần bằng Acc@5), còn SemLoc xếp hạng nhiều dòng liền nhau, nên khi nhãn lệch một dòng SemLoc vẫn "trúng" nhờ dòng hàng xóm.

5.2 RQ2 – LLM THẬT SỰ CẦN GÌ?

5.2.1 Một prompt có kiểm thử thất bại so với quy trình nhiều bước

Bảng 5.3 là kết quả chính trên ba bộ nhãn. **LLM + kiểm thử thất bại** — phương pháp đề xuất — thắng SemLoc trên cả ba bộ ở Acc@1: +7,7 [+3,5; +12,4] trên bộ gốc, +10,2 [+5,3; +16,0] trên bộ đã sửa và +12,5 [+8,0; +17,5] trên bộ đã rà soát. Ở Acc@3, nó hơn +14,5 [+8,5; +20,8] điểm trên bộ đã sửa và +10,0 [+6,3; +14,3] trên bộ đã rà soát; trên toàn bộ nhãn gốc, mức hơn +7,2 có khoảng tin cậy chạm 0, vì nhãn gốc chứa các lỗi mà không kiểm thử nào chạy tới (chỉ tính lỗi được chạy tới: +13,7 [+6,9; +20,4]). Phương pháp đề xuất tốn 2,0 nghìn token mỗi chương trình, so với 14,0 nghìn của SemLoc. SBFL thấp nhất trên mọi bộ nhãn.

Bảng 5.3: Kết quả chính trên ba bộ nhãn (tập test, Gemini 3.8 Flash, trung bình 3 lần chạy) và chi phí token mỗi chương trình

	Bản gốc ($n = 200$)		Đã sửa ($n = 179$)		Đã rà soát ($n = 200$)		
Phương pháp	Acc@1	Acc@3	Acc@1	Acc@3	Acc@1	Acc@3	nghìn token
SBFL Ochiai	3,0	15,5	3,9	19,0	11,0	28,0	0
LLM hỏi thẳng (chỉ có mã)	40,0	43,0	56,8	62,8	87,3	90,8	2,5
SemLoc (nhiều bước)	36,8	60,3	52,3	75,4	83,0	90,0	14,0
LLM + kiểm thử thất bại (nhóm đề xuất)	44,5	67,5	62,6	89,9	95,5	100,0	2,0
+ độ phủ từng dòng	40,2	68,2	57,2	92,0	92,2	100,0	2,6

Trên bộ đã rà soát, LLM + kiểm thử thất bại đạt Acc@3 100,0%: khi nhãn đúng, SemFault-250 gần như *bão hòa* với một prompt đơn giản. Chương 6 bàn vì sao.

5.2.2 Thông tin nào tạo ra khác biệt

Ablation tách hai nguồn thông tin trong prompt. **Kiểm thử thất bại** (tên kiểm thử và dòng thông báo lỗi đầu tiên) là tín hiệu chính: so với LLM hỏi thẳng, nó thay đổi Acc@3 +27,2 [+18,6; +37,0] điểm trên bộ đã sửa và +24,5 [+16,7; +33,3] trên bộ gốc. **Độ phủ**

từng dòng không giúp: thêm nó vào prompt thay đổi Acc@3 +2,1 [−0,2; +4,9] và làm Acc@1 đi xuống −5,4 [−9,4; −2,1] điểm trên bộ đã sửa; trên bộ đã rà soát Acc@1 cũng đi xuống (−3,3 [−5,9; −1,2]). Giả thuyết H3 bị bác bỏ: LLM cần biết *kiểm thử nào thất bại và vì sao*, không cần biết dòng nào được chạy qua.

5.2.3 Các cách bổ sung không cải thiện được

Bảng 5.4 tổng hợp mọi cách bổ sung nhóm đã thử, mỗi cách so với phương pháp nó mở rộng.

Bảng 5.4: Các cách bổ sung: chênh lệch so với phương pháp được mở rộng (ba dòng đầu trên tập test, 3 lần chạy, có khoảng tin cậy; hai dòng sau trên tập dev)

		Đã sửa		Đã rà soát	
Phần bổ sung (so với)	Tập	ΔAcc@1	ΔAcc@3	ΔAcc@1	ΔAcc@3
Độ phủ từng dòng (so với LLM + kiểm thử thất bại)	test	-5,4 [-9,4; -2,1]	+2,1 [-0,2; 4,9]	-3,3 [-5,9; -1,2]	+0,0 [0,0; 0,0]
CovGate (so với SemLoc)	test	+0,0 [0,0; 0,0]	+3,9 [1,6; 6,8]	+0,0 [0,0; 0,0]	+1,2 [0,0; 2,8]
Đề xuất rồi kiểm chứng, cấu hình được chọn (so với LLM + kiểm thử thất bại)	dev	+0,0	+0,0	+0,0	+0,0
Đề xuất rồi kiểm chứng, cấu hình mạnh tay nhất (so với LLM + kiểm thử thất bại)	dev	-4,3	+0,0	-11,3	+0,0
Biểu quyết 3 câu trả lời (so với LLM + kiểm thử thất bại)	dev	+0,0	-4,3	+1,3	+0,7

CovGate được chọn trên tập dev là một bước phá hòa bằng Ochiai; nó thay đổi Acc@3 +3,9 [+1,6; +6,8] điểm trên bộ đã sửa và không đổi Acc@1. Các thành phần còn lại còn làm kết quả đi xuống: "chỉ thưởng khi có căn cứ" thay đổi Acc@1 −4,1 điểm, xếp lại bằng kiểm chứng phản thực tế −2,4 điểm.

Đề xuất rồi kiểm chứng: cấu hình được chọn trên tập dev cho đúng thứ hạng của bước đề xuất (Acc@1 90,7% so với 90,7% trên bộ đã rà soát) — tức là cấu hình tốt nhất là cấu hình gần như không đổi gì; cấu hình mạnh tay nhất làm Acc@1 xuống 79,3%. Nguyên nhân nằm ở chính kiểm thử: trên bộ đã rà soát, bản sửa nhỏ làm mọi kiểm thử đạt ở 42,5% lần kiểm tại dòng lỗi thật (lỗi hai dòng, lỗi thiếu mã, lỗi cần sửa lớn hơn), nhưng cũng ở 23,3% lần kiểm tại dòng *không* có lỗi — ví dụ đổi một hằng số ở đầu tệp để bù cho phép tính sai ở dưới. Hai tỉ lệ không cách nhau đủ xa để làm tiêu chí xếp hạng.

Vì kết quả trên dev đã rõ, phương pháp này không được chạy trên tập test.

Biểu quyết ba câu trả lời của LLM + kiểm thử thất bại cho Acc@1 92,0% so với trung bình 90,7% của một câu trả lời (bộ đã rà soát, tập dev) — không khác biệt đáng kể.

5.2.4 Phân tích tĩnh

Bảng 5.5 cho thấy công cụ phân tích tĩnh thông dụng gần như không chỉ ra được dòng lỗi ngữ nghĩa. Ruff với luật mặc định có cảnh báo đúng dòng lỗi ở 1 trên 250 chương trình; bật mọi luật, nó cảnh báo 4172 lần và trúng 25 chương trình — *ít hơn* mức kỳ vọng 56,3 nếu các cảnh báo được rải ngẫu nhiên, vì cảnh báo phong cách tập trung ở vòng lặp và khối điều kiện dài trong khi lỗi thường nằm ở một phép so sánh hay phép tính ngắn. Pyrefly báo 21 lỗi kiểu, không lỗi nào đúng dòng lỗi; một số là *triệu chứng* (trả về số thực ở hàm khai báo trả về số nguyên) cách lỗi vài dòng. Kết quả chỉ nói về các công cụ này trên bộ dữ liệu này: chúng kiểm tra cách mã được viết, còn lỗi ngữ nghĩa là chỗ mã làm khác ý định.

Bảng 5.5: Phân tích tĩnh trên 250 chương trình (bộ nhãn đã rà soát)

Công cụ	Số cảnh báo	Số chương trình bị cảnh báo	Trúng dòng lỗi	Trúng do ngẫu nhiên (kỳ vọng)
ruff (luật mặc định)	148	109	1	2,4
ruff (mọi luật)	4172	250	25	56,3
pyrefly (kiểm tra kiểu)	21	21	0	0,6

5.2.5 Grounded hay guessed: giá trị biên và phân rã

Vì điểm thưởng lớn hơn mọi điểm ràng buộc, thêm phổ vào dòng đoán *không thể* thay đổi dòng đầu tiên: riêng dòng đoán và SemLoc cùng đạt Acc@1 52,3% trên bộ đã sửa, còn riêng phổ đạt 43,2%; trên bộ đã rà soát là 83,0% so với 56,8%. Giá trị Shapley của phổ ở Acc@1 (Bảng 5.6, 21,6 điểm) vì vậy là phần công cho điều phổ *sẽ* đạt được nếu không có dòng đoán, không phải đóng góp vào dòng đầu của SemLoc. Ở top-3, phổ có

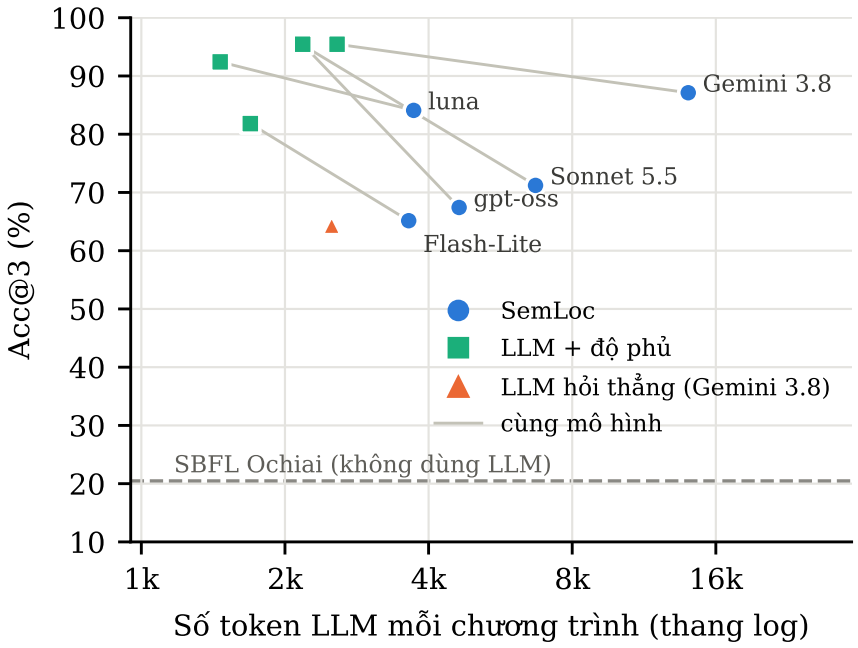
tác dụng thật: dòng đoán chỉ nêu một dòng nên dừng ở 52,3%, riêng phổ đạt 70,8%, và cả hai cùng đạt 75,4%. Kết luận: **hạng 1 là "guessed" theo thiết kế; bằng chứng thực thi của SemLoc có tác dụng ở top-3** — nhưng một prompt có kiểm thử thất bại đạt top-3 cao hơn mà không cần phổ. Trên nhật ký của tác giả (Claude), phân rã cho 42,8% = 27,8 (dòng đoán) + 15,0 (phổ).

Bảng 5.6: Phân rã Shapley độ chính xác của SemLoc (lượt chạy của nhóm, 3 lần)

Dòng lỗi chuẩn	Chỉ số	Tổng	Dòng LLM đoán	Phổ ngữ nghĩa	Độ phủ	CF
SemLoc như phát hành (dòng đoán + phổ)						
Bản gốc ($n = 200$)	Acc@1	36,8	22,2 [16,5; 28,4]	14,6 [11,8; 17,5]	–	–
Bản gốc ($n = 200$)	Acc@3	60,3	19,8 [16,1; 23,8]	40,6 [35,6; 45,3]	–	–
Đã sửa ($n = 179$)	Acc@1	52,3	30,7 [23,5; 37,8]	21,6 [17,4; 26,1]	–	–
Đã sửa ($n = 179$)	Acc@3	75,4	28,5 [23,4; 33,5]	46,9 [41,9; 51,7]	–	–
Mọi tín hiệu (+ Ochiai thô, + xếp lại bằng CF)						
Bản gốc ($n = 200$)	Acc@1	35,0	22,7 [17,4; 28,3]	12,5 [10,0; 15,2]	-0,5 [-1,5; 0,6]	0,3 [-0,6; 1,1]
Bản gốc ($n = 200$)	Acc@3	59,0	17,6 [14,2; 21,4]	35,8 [31,6; 40,2]	4,6 [0,1; 9,0]	1,1 [0,5; 1,8]
Đã sửa ($n = 179$)	Acc@1	49,9	31,9 [25,3; 38,5]	18,6 [14,4; 22,9]	-1,1 [-2,3; 0,2]	0,5 [-0,7; 1,5]
Đã sửa ($n = 179$)	Acc@3	80,6	25,9 [20,8; 31,0]	44,4 [39,7; 49,3]	9,4 [5,8; 13,4]	0,9 [0,3; 1,7]

5.2.6 Chi phí theo mô hình

Hình 5.1 đặt độ chính xác cạnh chi phí cho năm mô hình trên cùng một tập con (44 chương trình), với phương pháp một prompt ở dạng có độ phủ (dạng được chạy trên mọi mô hình): với mọi mô hình, nó có Acc@3 cao hơn và tốn ít token hơn SemLoc. Các so sánh theo từng mô hình chưa có khoảng tin cậy; ở Acc@1, GPT-6 luna không hơn.



Hình 5.1: Chi phí và độ chính xác (Acc@3) trên cùng tập con, dòng lỗi chuẩn đã sửa; đường nối hai phương pháp của cùng một mô hình

5.3 RQ3 – ĐỘ BỀN

5.3.1 Lỗi thật BugsInPy

Bảng 5.7 cho kết quả trên 31 lỗi của một dự án (youtube-dl), ba lần chạy. Theo thiết kế, dòng đầu của SemLoc là dòng LLM đoán, nên Acc@1 của SemLoc chính là của dòng đoán (46,2%); riêng phổ ngữ nghĩa chỉ đặt đúng 7,5% ở hạng 1, gần SBFL (9,7%). LLM + kiểm thử thất bại đi cùng chiều với kết quả trên SemFault-250 — Acc@3 71,0% so với 53,8% — và thêm độ phủ cũng không giúp (69,9%); nhưng với 31 lỗi, khoảng tin cậy [−3,2; +36,6] vẫn chứa 0. Trên lỗi thật, mọi phương pháp còn xa trần: đây là nơi còn chỗ để cải tiến. “Lỗi được chạy tới” là các lỗi có kiểm thử thất bại chạy qua một dòng lỗi chuẩn; chỉ số của gói mã công bố (dòng cuối bảng) là hạng của dòng LLM đoán, đo mức nhất quán của LLM với chính nó chứ không đo khả năng định vị.

Bảng 5.7: RQ3a: 31 lỗi thật youtube-dl (BugsInPy), 3 lần chạy

	Mọi lỗi ($n = 31$)			Lỗi được chạy tới ($n = 26$)	
Phương pháp	Acc@1	Acc@3	Acc@5	Acc@1	Acc@3
Chỉ phổ ngữ nghĩa của SemLoc	7,5±1,9	29,0±3,2	37,6±3,7	9,0±2,2	34,6±3,9
SemLoc (phổ + điểm thưởng dòng đoán)	46,2±6,7	53,8±7,4	55,9±6,7	51,3±8,0	60,3±8,9
Chỉ dòng LLM đoán	46,2±6,7	46,2±6,7	46,2±6,7	51,3±8,0	51,3±8,0
CovGate (nhóm đề xuất)	46,2±6,7	54,8±6,5	57,0±7,4	51,3±8,0	61,5±7,7
LLM + kiểm thử thất bại (1 prompt)	50,5±1,9	71,0±3,2	77,4±3,2	56,4±2,2	71,8±2,2
LLM + độ phủ (1 prompt, nhóm đề xuất)	51,6±5,6	69,9±3,7	75,3±1,9	59,0±5,9	75,6±4,4
SBFL Ochiai	9,7±0,0	29,0±0,0	29,0±0,0	11,5±0,0	34,6±0,0
SBFL DStar	9,7±0,0	29,0±0,0	29,0±0,0	11,5±0,0	34,6±0,0
Chỉ số của gói mã công bố (hạng của dòng LLM đoán)	39,8±1,9	45,2±3,2	51,6±3,2	37,2±2,2	43,6±5,9

5.3.2 Đổi tên định danh

Bảng 5.8 so sánh từng chương trình với bản đổi tên của nó, theo cặp lần chạy (chỉ với Gemini 3.8 Flash). Dòng LLM đoán không thay đổi đáng kể (Acc@1 $\Delta = -2{,}6$ [−7,8; +2,4]), nên không có dấu hiệu mô hình đoán nhờ nhớ tên. Ngược lại, phổ vi phạm ràng buộc cần tên có nghĩa: không có điểm thưởng, Acc@3 thay đổi −15,5 điểm. Phương pháp một prompt (dạng có độ phủ; dạng không độ phủ chưa được chạy trên bản đổi tên) không thay đổi đáng kể: Acc@3 $\Delta = -0{,}9$ [−3,5; +1,3]. Lưu ý đổi tên không đổi tên kiểm thử, nên tên kiểm thử vẫn mô tả ý định.

Bảng 5.8: RQ3b: bản gốc và bản đổi tên định danh (dòng lỗi chuẩn đã sửa, 3 cặp lần chạy)

	Acc@1			Acc@3		
Phương pháp	Bản gốc	Đổi tên	Δ [CI 95%]	Bản gốc	Đổi tên	Δ [CI 95%]
SemLoc	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]	75,3±2,0	66,8±2,0	-8,4 [-14,1; -3,0]
bỏ điểm thưởng dòng đoán	42,9±2,1	34,5±2,1	-8,4 [-13,5; -3,7]	70,6±2,3	55,1±2,6	-15,5 [-21,4; -10,3]
Chỉ dòng LLM đoán	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]	52,1±0,9	49,4±1,0	-2,6 [-7,8; 2,4]
LLM + độ phủ (1 prompt)	56,9±0,9	56,2±0,0	-0,8 [-3,6; 2,0]	92,0±0,3	91,0±1,5	-0,9 [-3,5; 1,3]

5.3.3 Các họ mô hình

Trên tập con (Bảng 5.9, chưa có khoảng tin cậy), mô hình yếu hơn dựa vào điểm thưởng nhiều hơn: bỏ điểm thưởng, Acc@1 của Gemini 3.5 Flash-Lite đi từ 42,4% xuống 20,4%, của gpt-oss-120b từ 45,5% xuống 29,5%. Với Gemini 3.8 Flash, tập con gần như không đổi (52,3% so với 53,0%) trong khi trên toàn tập test Acc@1 thay đổi −9,1 điểm; tập con quá nhỏ để xếp hạng các mô hình theo hiệu ứng này.

Bảng 5.9: RQ3c: năm mô hình trên cùng tập con (dòng lỗi chuẩn đã sửa, 3 lần chạy)

		SemLoc		bỏ điểm thưởng dòng đoán		CovGate
Mô hình	Số lần lặp	Acc@1	Acc@3	Acc@1	Acc@3	Acc@3
Gemini 3,8 Flash	3	52,3±2,3	87,1±1,3	53,0±4,7	84,1±0,0	87,9±2,6
GPT-6 luna	3	53,8±2,6	84,1±2,3	40,9±6,0	72,7±2,3	84,9±1,3
Claude Sonnet 5,5	3	43,2±2,3	71,2±1,3	37,9±1,3	68,9±1,3	71,2±1,3
gpt-oss-120b	3	45,5±6,0	67,4±3,5	29,5±7,9	57,6±6,6	68,2±2,3
Gemini 3,5 Flash-Lite	3	42,4±3,5	65,1±5,7	20,4±2,3	51,5±4,7	71,2±1,3

5.3.4 Ba bộ nhãn

Bảng 5.3 đã trả lời phần (d): trên cả ba bộ nhãn, LLM + kiểm thử thất bại thắng SemLoc ở Acc@1, độ phủ không thêm được gì (Bảng 5.4), và bỏ điểm thưởng làm Acc@1 của SemLoc đi xuống. Thứ duy nhất đổi chiều theo bộ nhãn là so sánh giữa SemLoc và LLM hỏi thẳng ở Acc@3 — và chính chiều thay đổi đó cho thấy nhãn lệch có lợi cho SemLoc.

5.4 TÓM TẮT TRẢ LỜI CÁC CÂU HỎI NGHIÊN CỨU

Bảng 5.10: Trả lời ba câu hỏi nghiên cứu

Câu hỏi	Trả lời
RQ1	Tính lại từ nhật ký khớp toàn bộ, chạy lại thì không; điểm thưởng ẩn quyết định dòng đầu; bước kiểm chứng phản thực tế không đổi thứ hạng; mã chấm điểm lạc quan với gỡ lỗi chia đôi và bi quan với SBFL; gần một nửa nhãn bị dời khi rà soát, và nhãn lệch có lợi cho SemLoc so với LLM hỏi thẳng.
RQ2	Một prompt có kiểm thử thất bại thắng quy trình nhiều bước ở Acc@1 trên cả ba bộ nhãn, rẻ hơn khoảng bảy lần; kiểm thử thất bại là tín hiệu chính, độ phủ từng dòng không giúp; CovGate, đề xuất rồi kiểm chứng, biểu quyết và phân tích tĩnh không cải thiện được. Hạng 1 của SemLoc là “guessed” theo thiết kế.
RQ3	Trên 31 lỗi thật, kết quả cùng chiều nhưng chưa đủ mẫu; đổi tên làm yếu phổ ràng buộc nhưng không ảnh hưởng đáng kể đến dòng đoán hay phương pháp một prompt; mô hình yếu dựa vào dòng đoán nhiều hơn (tập con, chưa có khoảng tin cậy); kết luận giữ trên cả ba bộ nhãn.

Chương 6. Phân tích thất bại và thảo luận

Chương này trả lời câu hỏi: khi SemLoc (hoặc dòng LLM tự đoán) sai, nó sai thế nào và vì sao? Phân tích dùng ba lần chạy Gemini 3.8 Flash trên tập test với bộ dòng lỗi chuẩn đã sửa; tín hiệu nhị phân của mỗi chương trình lấy theo đa số ba lần.

6.1 PHÂN NHÓM TỰ ĐỘNG

6.1.1 Sáu nhóm kết quả

Mỗi chương trình được xếp vào đúng một nhóm theo thứ tự ưu tiên của Bảng 6.1. Trên 179 chương trình, khoảng một nửa được giải quyết bởi chính dòng LLM tự đoán; 25 chương trình là nơi bằng chứng thực thi thật sự "cứu" SemLoc khi dòng đoán sai — đây là phiên bản theo từng chương trình của phân rã Shapley ở mục 5.2.

Bảng 6.1: Phân nhóm kết quả theo chương trình (dòng lỗi chuẩn đã sửa)

Nhóm	Định nghĩa	Số chương trình
`gt_suspect`	dòng lỗi chuẩn chưa kiểm chứng được và mọi phương pháp trượt top-3	6
`guess_right`	dòng LLM tự đoán là dòng lỗi chuẩn	94
`prior_hurts`	phổ xếp dòng lỗi hạng 1 nhưng điểm thưởng đẩy dòng đoán sai lên trên	19
`rescued_by_spectrum`	đoán sai nhưng SemLoc vẫn có dòng lỗi trong top-3	25
`coverage_only_wins`	SemLoc trượt top-3, LLM + độ phủ trúng top-3	32
`all_miss`	còn lại: mọi phương pháp trượt	3

6.1.2 Cách chọn ca để đọc

Trong mỗi nhóm trừ `guess_right`, nhóm chọn tối đa bốn ca, ưu tiên các cụm khuôn mẫu khác nhau, với hạt giống cố định, để mẫu đọc trải rộng chứ không lặp lại một khuôn mẫu. Với mỗi ca, công cụ trích đoạn mã quanh dòng lỗi chuẩn và dòng đoán, dòng đoán của từng lần chạy, năm dòng đầu của SemLoc và của LLM + độ phủ, và ràng buộc có điểm cao nhất.

6.2 MƠ HỒ DÒNG KỀ CẠNH

6.2.1 Dòng đoán lệch có hệ thống xuống dưới

Trên mọi lần đoán của ba lần chạy, dòng đoán trùng dòng lỗi chuẩn 281 lần; lệch xuống một dòng 99 lần và hai dòng 61 lần, nhưng lệch lên một dòng chỉ 28 lần và hai dòng 26 lần. LLM + độ phủ lệch cùng chiều (113 lần xuống một dòng so với 24 lần lên một dòng). Phần lớn các lần "sai" vì vậy không phải chỉ vào một chỗ khác, mà chỉ vào dòng ngay bên dưới.

6.2.2 Quan hệ đầu khối – thân khối và độ nhạy của Acc@1

Phân tích cây cú pháp cho thấy, trong 85 chương trình mà dòng đoán sai, 44 ca có dòng đoán và dòng lỗi chuẩn là *đầu một câu lệnh ghép* (`if`, `for`, `while`…) *và câu lệnh mà nó bảo vệ*. Nếu chấp nhận cả hai dòng của cặp này là đúng, Acc@1 của SemLoc tăng từ 52,3% lên 73,9% (biến thể chặt: chỉ dòng đầu khối và dòng đầu tiên của thân) đến 76,2% (cận trên); của LLM + độ phủ từ 57,2% lên 79,3%–81,6%. Hơn hai mươi điểm Acc@1 phụ thuộc vào quy ước gán nhãn — lớn hơn mọi khác biệt giữa các phương pháp dùng LLM ở Chương 5; thứ tự giữa các phương pháp không đổi.

6.3 NĂM MẪU NGUYÊN NHÂN

6.3.1 Dòng điều kiện và câu lệnh được bảo vệ

Mẫu phổ biến nhất (sáu trong mười chín ca đọc): dòng lỗi chuẩn là dòng điều kiện, còn lỗi cài vào nằm ở câu lệnh nó bảo vệ — hoặc ngược lại. Ví dụ trong chương trình `bump_click`:

```
    if cap is not None and counters[key] > cap:    # dòng lỗi chuẩn
        counters[key] = cap - 1                    # LLM chỉ dòng này
```

Lỗi cài vào là `- 1`; LLM đúng về ngữ nghĩa nhưng sai một dòng so với nhãn. Các ca tương tự: `ledger_balance_rollup` (`elif kind == "debit":` và `balance += amount`), `quota_for_fuel` (`if _INTEGER_ALLOCATION:` và phép cắt `int(x)`), và chiều ngược lại ở `gain_trendline`, `estimate_supply`, nơi dòng lỗi chuẩn nằm trong thân khối còn LLM chỉ chính điều kiện bị đảo `if not clamp:`.

6.3.2 Khối lỗi nhiều dòng

Ba ca có lỗi cài vào là cả một khối `if … merged[-1] == left[-1] == right[-1]:` `merged.pop()`; xóa bất kỳ dòng nào của khối cũng sửa được lỗi. Nhãn chỉ nêu một dòng, LLM chỉ dòng kia — và chính các ca này bị xếp vào `prior_hurts`. Như vậy, phần lớn trường hợp "điểm thưởng gây hại" thật ra là hai dòng tương đương về mặt sửa lỗi.

6.3.3 Bộ phán định chấp nhận bản sửa bù trừ

Bộ phán định sửa tối thiểu của nhóm (mục 4.2) chỉ yêu cầu mọi kiểm thử đạt, nên đôi khi chấp nhận một bản sửa *bù trừ*. Ở `gradient_clip_budget`, dòng chuẩn gốc là dòng trống và được "sửa" thành dòng tính chuẩn bằng `norm = _compute_global_norm(gradients) - 1`, trong khi lỗi thật là `max_norm / (norm + 1)` ở dòng mà

LLM chỉ cả ba lần. Ở `energy_window_avg`, bộ phán định "xác nhận" bằng cách thay một lời gọi ghi log bằng một phép gán. Ở các khối lỗi nhiều dòng, bộ phán định "xác nhận" bằng cách làm điều kiện luôn sai. Các bản sửa đặt trên dòng trống đã bị loại khỏi bộ dòng lỗi chuẩn đã sửa (13 chương trình); bản sửa bù trừ trên dòng có mã thì không phát hiện tự động được, nhưng vòng rà soát (mục 4.2) đã bác 8 trong 46 lần bộ phán định dời nhãn.

6.3.4 Nguyên nhân và triệu chứng trên cùng luồng dữ liệu

Bốn ca thuộc mẫu này: dòng lỗi chuẩn là nơi tính một giá trị, LLM chỉ nơi dùng giá trị đó, hoặc ngược lại (`allocate_bounty_cut`: trừ phí trên từng phần hay tính phần chia); hoặc lỗi là *thiếu* một bước chuẩn hóa (`shift_trust`, `shift_risk`) hay thiếu một lệnh lưu (`warm_cache_loader`), không có "dòng sai" duy nhất. Đọc mã không đủ để phân xử; cần bộ phán định thực thi kiểm tra cả hai dòng.

6.3.5 Dòng lỗi chuẩn đáng ngờ

Bốn ca còn lại có dòng lỗi chuẩn khó bảo vệ: `reserve_slot` và `schedule_call` gán lỗi cho dòng khai báo một hàm lồng (`def overlaps(…)`); `file_chunk_stitch` gán cho `data += part` trong khi phép cắt `return data[:-1]` ngay dưới mới là lỗi rõ ràng; `room_reservation` có các lần chạy không thống nhất.

6.4 THẢO LUẬN

6.4.1 Đọc "guessed" cho đúng

Kết luận "hạng 1 của SemLoc là dòng LLM đoán" của Chương 5 vẫn đúng, nhưng phân tích thất bại cho thấy dòng đoán thường *đúng về ngữ nghĩa nhưng lệch một dòng* so với nhãn. Vì vậy không thể suy ra "bỏ điểm thưởng thì tốt hơn": các ca điểm thưởng "gây

hại" gần như toàn là cặp đầu khối – thân khối hoặc khối lỗi nhiều dòng. Điều thật sự thiếu là một cách đánh giá chấp nhận các vị trí sửa tương đương.

6.4.2 Khi nào bằng chứng thực thi có ích

Bằng chứng thực thi có ích rõ ràng ở top-3: phổ vi phạm đưa dòng lỗi vào ba dòng đầu ở 25 chương trình mà dòng đoán sai, và bằng chứng đưa thẳng cho LLM còn mạnh hơn (32 chương trình SemLoc trượt top-3 nhưng phương pháp một prompt trúng). Ablation ở mục 5.2 cho biết phần "bằng chứng" có ích là kiểm thử thất bại, không phải độ phủ. Bằng chứng không giúp phân biệt một dòng đoán đúng với một dòng đoán sai ở hạng 1. Bài học cho thiết kế công cụ: đưa thẳng kiểm thử thất bại cho LLM, thay vì để LLM đoán rồi cộng điểm thưởng.

6.4.3 Vì sao kiểm thử không làm trọng tài tốt cho vị trí lỗi

Ba hiện tượng trong tiểu luận có chung một nguyên nhân. Bước vá thử của SemLoc không đổi thứ hạng nào; bộ phán định sửa tối thiểu dời nhầm 8 nhãn; và "đề xuất rồi kiểm chứng" không cải thiện được bước đề xuất. Cả ba dùng cùng một tiêu chí: *một bản sửa nhỏ tại dòng này làm mọi kiểm thử đạt*. Với khoảng mười kiểm thử mỗi chương trình, tiêu chí này thỏa được ở 23,3% lần thử tại dòng không có lỗi — thường bằng cách đổi một hằng số hay thêm một phép trừ để bù cho phép tính sai ở chỗ khác — trong khi chỉ thỏa được ở 42,5% lần thử tại dòng lỗi thật, vì lỗi hai dòng, lỗi thiếu mã hay lỗi cần sửa nhiều từ tố không sửa được bằng một thay đổi nhỏ trên một dòng. Đây đúng là hiện tượng "bản vá hợp lệ bề ngoài" [30, 31] của sửa lỗi tự động, nhìn từ phía định vị lỗi: bộ kiểm thử đủ để *gợi ý* lỗi nằm ở đâu, nhưng quá yếu để *chứng minh* lỗi nằm ở đâu.

6.4.4 Bộ dữ liệu bão hòa khi nhãn đúng

Trên nhãn đã rà soát, một prompt có kiểm thử thất bại đạt Acc@3 100,0%. Ba đặc điểm của SemFault-250 giải thích điều này: chương trình là một hàm ngắn mà phần tính toán thật chỉ vài dòng; lỗi được tiêm bằng một thay đổi nhỏ (đổi phép so sánh, thêm `- 1`, đảo điều kiện); và tên kiểm thử mô tả gần như trực tiếp ý định bị vi phạm (ví dụ `test_fee_deducted_once_not_per_participant`). Hệ quả là trên bộ dữ liệu này không còn đủ khoảng trống để chứng minh một phương pháp phức tạp hơn là cần thiết; "độ khó" trong bài báo gốc phần lớn đến từ nhãn lệch. Trên lỗi thật (mục 5.3), mọi phương pháp còn xa trần.

6.4.5 Hàm ý cho công cụ trong thực tế

Các trợ lý và tác tử AI sửa lỗi — chạy trên máy cá nhân hay trong CI — thường dừng khi bản sửa làm kiểm thử đạt. Kết quả trên gợi ý thận trọng: trên một bộ kiểm thử cỡ mười ca, khoảng một phần tư số dòng không có lỗi vẫn "sửa được", nên một bản sửa vượt kiểm thử cần được người xem lại, hoặc cần kiểm thử chặt hơn trước khi coi là đúng. Ngược lại, thông tin đáng đầu tư nhất cho định vị lỗi là kiểm thử viết rõ ý định — đúng thứ phát triển hướng kiểm thử tạo ra.

Chương 7. Đóng góp, triết lý khoa học và các mối đe dọa

7.1 CÁC ĐÓNG GÓP KHOA HỌC

Mỗi đóng góp dưới đây được trình bày dưới dạng: khẳng định, bằng chứng (tệp dữ liệu trong kho mã), điều kiện sẽ bác bỏ nó, và giới hạn.

7.1.1 C1. Hạng 1 của SemLoc là dòng LLM đoán

Khẳng định. Theo thiết kế, dòng hạng 1 của SemLoc luôn là dòng LLM tự đoán, nhờ một điểm thưởng không được mô tả trong bài báo; bằng chứng thực thi của SemLoc chỉ có tác dụng từ hạng 2 trở đi. Acc@1 của SemLoc (52,3% trên bộ đã sửa) là của dòng đoán; riêng phổ vi phạm đạt 43,2%; trên lỗi thật, Acc@1 của SemLoc bằng đúng Acc@1 của riêng dòng đoán (46,2%). Bước vá thử chiếm 42% lượt gọi LLM nhưng không đổi thứ hạng nào. **Bằng chứng:** `data/rq_decomposition.csv`, `data/rq3_bugsinpy.csv`, `data/rq1_cf_identity.json`. **Bác bỏ nếu:** bỏ điểm thưởng mà Acc@1 không đi xuống; ở đây nó thay đổi −9,1 [−16,7; −1,9] điểm (bộ đã sửa) và −26,2 điểm (bộ đã rà soát). **Giới hạn:** giá trị Shapley phụ thuộc cách định nghĩa "tắt" một tín hiệu.

7.1.2 C2. Gần một nửa nhãn của SemFault-250 không trỏ đúng dòng lỗi

Khẳng định. 112 trong 250 nhãn gốc bị dời khi rà soát (56 trên dòng trống, 32 lệch một dòng); nhãn lệch có lợi cho SemLoc so với LLM hỏi thẳng. Nhóm công bố hai bộ nhãn: *đã sửa* (chỉ kiểm thử quyết định, 82 xác nhận, 46 dời vị trí) và *đã rà soát* (LLM rà trước, 6 thành viên duyệt 111 nhãn, 5 ca sửa có bằng chứng chạy kiểm thử). **Bằng chứng:** `data/label_audit.csv`, `data/semfault_verified_gt.json`, `data/semfault_reviewed_gt.json`, `docs/analysis/gt-review/`. **Giới hạn:** người duyệt có xem gợi ý của LLM và đồng ý ở 111/111 ca, nên bộ đã rà soát không

độc lập với LLM; mọi kết quả chính được báo cáo trên cả bộ gốc và bộ đã sửa.

7.1.3 C3. Một prompt có kiểm thử thất bại thắng quy trình nhiều bước

Khẳng định. LLM + kiểm thử thất bại hơn SemLoc ở Acc@1 trên cả ba bộ nhãn (+10,2 [+5,3; +16,0] trên bộ đã sửa) và ở Acc@3 trên bộ đã sửa và bộ đã rà soát, với 2,0 nghìn token mỗi chương trình so với 14,0 nghìn. **Bằng chứng:** `data/rq2_ours.csv`, `data/llm_cost.csv`. **Bác bỏ nếu:** khoảng tin cậy của chênh lệch so với SemLoc nằm hẳn dưới 0 trên một bộ nhãn nào đó. **Giới hạn:** một mô hình chính; mỗi lần chỉ xét một hàm; trên 31 lỗi thật kết quả cùng chiều nhưng khoảng tin cậy còn chứa 0; prompt do nhóm thiết kế.

7.1.4 C4. Thêm phức tạp không thêm được gì — và vì sao

Khẳng định. Trong prompt, kiểm thử thất bại là tín hiệu chính (Acc@3 +27,2 so với LLM hỏi thẳng); độ phủ từng dòng không giúp ở Acc@3 và làm Acc@1 đi xuống. CovGate, đề xuất rồi kiểm chứng và biểu quyết không cải thiện được phương pháp chúng mở rộng; ruff và pyrefly gần như không trúng dòng lỗi. Cơ chế chung: một bản sửa nhỏ làm mọi kiểm thử đạt ở 23,3% lần thử tại dòng không có lỗi, nên kiểm thử không làm được trọng tài cho vị trí lỗi. **Bằng chứng:** `data/ablation_coverage.csv`, `data/pav_dev_grid.csv`, `data/pav_mechanism.csv`, `data/vote_dev.csv`, `data/static_tools.csv`. **Giới hạn:** đề xuất rồi kiểm chứng và biểu quyết chỉ được đánh giá trên tập dev; kết quả âm tính nói về các cách đã thử trên bộ dữ liệu này, không chứng minh mọi cách bổ sung đều vô ích.

7.1.5 C5. Bảng kiểm tra đánh giá và gói tái hiện

Bảng kiểm tra bảy mục (phá hòa, mẫu số, dòng lỗi kiểm chứng, loại bỏ thành phần ẩn, lặp và kiểm định, chi phí, so với một prompt có kiểm thử thất bại) được áp cho mọi phương

pháp. Gói tái hiện gồm Docker chạy không cần mạng, toàn bộ nhật ký và bộ đệm lượt gọi LLM để phát lại, tệp duyệt nhãn của từng thành viên, và macro sinh tự động cho mọi con số trong báo cáo.

7.2 TRIẾT LÝ KHOA HỌC

7.2.1 Bằng chứng thực thi là trọng tài — và giới hạn của nó

Ở mọi chỗ cần phán xử, nhóm ưu tiên để kiểm thử quyết định và để LLM chỉ sinh ứng viên; khi kiểm thử không đủ để quyết định (nhãn của lỗi hai dòng, lỗi thiếu mã, bản sửa bù trừ), nhóm dùng thêm người duyệt và nói rõ phần nào do ai quyết định. Đây cũng chính là câu hỏi đặt cho SemLoc: nếu một phương pháp tuyên bố neo lập luận vào thực thi, phần neo đó phải được đo. Tái hiện không phải là chép lại con số: khớp số công bố là điều kiện cần; điều kiện đủ là hiểu vì sao ra con số đó và kết quả còn đứng vững khi đổi cách chấm hợp lý.

7.2.2 Đo đóng góp, so với phương pháp đối chứng đơn giản, và chấp nhận bị bác bỏ

Một con số Acc@1 không nói nó đến từ đâu; phân rã đóng góp và loại bỏ từng thành phần biến câu hỏi "phương pháp có tốt không" thành "phần nào làm nó tốt". Một quy trình nhiều bước chỉ đáng giá khi thắng cách đơn giản mà người làm nghề vẫn dùng — ở đây là đưa kiểm thử thất bại cho LLM. Kiểm thử là trọng tài tốt cho *việc chương trình có đúng không*, nhưng chỉ là trọng tài tốt cho *chỗ sai nằm ở đâu* khi bộ kiểm thử đủ chặt; nhóm báo cáo cả các kết quả âm tính cho thấy giới hạn đó. Mỗi khẳng định đi kèm điều kiện bác bỏ và khoảng tin cậy; đơn vị lấy mẫu là chương trình (hoặc cụm khuôn mẫu), không coi các lần chạy là mẫu độc lập; kết quả được báo cáo cả khi khoảng tin cậy chứa 0. Cuối cùng, bộ dữ liệu cũng là một giả thuyết: nhãn có thể sai hoặc mơ hồ, và cả bộ

phán định sửa nhãn lẫn vòng rà soát của nhóm đều có giới hạn cần nêu rõ.

7.3 CÁC MỐI ĐE DỌA ĐẾN TÍNH HỢP LỆ

7.3.1 Tính hợp lệ nội tại và tính hợp lệ của cấu trúc

- **Bộ phán định sửa tối thiểu** chấp nhận bản sửa bù trừ nên bộ dòng lỗi đã sửa còn nhiễu; mọi kết quả cũng được báo cáo trên bộ gốc.
- **Bộ dòng lỗi kiểm chứng có thể thiên về phương pháp dùng độ phủ**, vì bộ phán định cần kiểm thử thất bại chạy tới dòng sửa; so sánh giữa SemLoc, các phương pháp đối chứng dùng LLM và CovGate ở Acc@1 và Acc@3 cùng chiều trên bộ gốc, riêng cổng độ phủ thì không; ở Acc@5, SemLoc hơn LLM + độ phủ trên bộ gốc nhưng kém trên bộ đã sửa.
- **Mơ hồ dòng kề cạnh**: Acc@1 chặt phụ thuộc quy ước gán nhãn; nhóm báo cáo cả Acc@1 chấp nhận đầu khối/thân khối.
- **Prompt của phương pháp một prompt** do nhóm viết trước khi xem kết quả và không tinh chỉnh; prompt khác có thể cho kết quả khác.
- **Vòng tròn của bộ nhãn đã rà soát**: nhãn do một LLM đề xuất và người duyệt có xem gợi ý, nên các phương pháp dùng LLM có thể "nghĩ giống" người ra đáp án; kết quả chính vì vậy được báo cáo trên bộ gốc và bộ đã sửa (không có ý kiến LLM).
- **Nhãn hai dòng** (44 chương trình) làm Acc@k trên bộ đã rà soát dễ hơn một chút.
- **Tên kiểm thử "nói hộ" lỗi**: kiểm thử của SemFault-250 mô tả rất rõ ý định; trên dự án có kiểm thử đặt tên chung chung, lợi thế của phương pháp một prompt có thể nhỏ hơn. Nhóm chưa chạy thí nghiệm giấu tên kiểm thử.

7.3.2 Tính hợp lệ ngoại tại

- **Lỗi thật ít và chỉ một dự án** (31 lỗi youtube-dl); hàm đích được chọn từ bản sửa của lập trình viên nên định vị cấp hàm — phần khó nhất trên dự án lớn — không được đánh giá; một lỗi còn hai kiểm thử thất bại do môi trường trong bộ chạy gốc của SemLoc.
- **Mô hình**: mô hình chính mới hơn mô hình trong bài gốc; mỗi nhà cung cấp chạy ở mức suy luận mặc định, nên so sánh giữa họ mô hình là "ở mặc định" chứ không cùng mức suy luận; Claude qua cổng trung gian sinh ràng buộc yếu hơn nhật ký công bố.
- **Đề xuất rồi kiểm chứng và biểu quyết** chỉ được đánh giá trên tập dev (50 chương trình); kết luận âm tính dựa trên mức chênh bằng 0 ở cấu hình được chọn và cơ chế đo được, không phải trên tập test.
- **Nhiễm dữ liệu**: biến thể đổi tên chỉ đổi định danh và xóa chú thích, cấu trúc mã giữ nguyên, nên không loại trừ hoàn toàn việc mô hình đã thấy các khuôn mẫu.

Chương 8. Kết luận và hướng phát triển

8.1 KẾT LUẬN

Tiểu luận đặt câu hỏi “grounded hay guessed?” cho SemLoc [1], phương pháp định vị lỗi ngữ nghĩa tuyên bố neo lập luận của LLM vào bằng chứng thực thi, và so nó với cách đơn giản mà lập trình viên vẫn làm: chạy kiểm thử, đưa kết quả thất bại cho LLM. Bằng cách tính lại từ nhật ký và chạy lại từ mã nguồn của tác giả, kiểm tra lại chính các nhãn của bộ dữ liệu, lặp mọi thực nghiệm ba lần với kiểm định thống kê, và tách đóng góp của từng tín hiệu, nhóm rút ra bốn kết luận.

Thứ nhất, kết quả công bố tính lại được từ nhật ký, nhưng dòng hạng 1 của SemLoc — theo một lựa chọn thiết kế không có trong bài báo — luôn là dòng LLM tự đoán, và bước vá thử không đổi thứ hạng nào. Thứ hai, gần một nửa nhãn của SemFault-250 (112 trên 250) không trỏ đúng dòng lỗi, và nhãn lệch vô tình có lợi cho SemLoc. Thứ ba, một prompt duy nhất có kiểm thử thất bại thắng quy trình nhiều bước ở Acc@1 trên cả ba bộ nhãn và rẻ hơn khoảng bảy lần; thứ LLM cần là kiểm thử thất bại, không phải độ phủ từng dòng. Thứ tư, mọi cách thêm phức tạp mà nhóm thử — gắn độ phủ vào SemLoc, kiểm chứng dòng nghi ngờ bằng bản sửa, biểu quyết — và các công cụ phân tích tĩnh đều không thêm được gì, vì kiểm thử của bộ dữ liệu đủ để gợi ý lỗi nhưng quá yếu để chứng minh vị trí lỗi. Các kết luận này đến từ một mô hình chính, các chương trình một hàm có kiểm thử mô tả rõ ý định, và 31 lỗi thật của một dự án. Phân tích thất bại cho thấy thêm rằng độ chính xác cấp dòng phụ thuộc vào việc nhãn được gán ở dòng điều kiện hay ở câu lệnh mà điều kiện bảo vệ, và khi nhãn đúng, bộ dữ liệu gần như bão hòa với một prompt đơn giản. Vì vậy, nhóm đề xuất mọi đánh giá định vị lỗi bằng LLM áp dụng bảng kiểm tra bảy mục: phá hòa bằng hạng trung bình; mẫu số cố định; dòng lỗi chuẩn được kiểm chứng; loại bỏ từng thành phần, kể cả thành phần không được mô tả; lặp và báo khoảng tin cậy ghép cặp; báo cáo chi phí; và so với một prompt duy nhất có kiểm thử thất bại.

8.2 HƯỚNG PHÁT TRIỂN

Về nghiên cứu, bốn hướng mở ra từ kết quả: (i) định vị lỗi cấp dự án — thu hẹp từ cả kho mã xuống một hàm trước khi hỏi LLM, phần khó nhất trên dự án lớn mà tiểu luận chưa đánh giá; (ii) một thí nghiệm giấu tên kiểm thử để đo phần thông tin mà tên kiểm thử "nói hộ"; (iii) bộ kiểm thử chặt hơn (ví dụ sinh thêm kiểm thử phân biệt các bản sửa) để kiểm chứng vị trí lỗi bằng thực thi có ý nghĩa; (iv) một chuẩn đánh giá trên lỗi thật của nhiều dự án, trong đó "một prompt có kiểm thử thất bại" là mốc so sánh bắt buộc.

Phụ lục A. Hướng dẫn tái hiện và bảng bổ sung

A.1 BỐN MỨC TÁI HIỆN

A.1.1 Từ dữ liệu đến bảng

Toàn bộ mã nguồn, dữ liệu dẫn xuất và hướng dẫn nằm tại `https://github.com/doanan293/grounded-or-guessed` (tệp `REPRODUCE.md`). Bốn mức, từ rẻ đến đắt:

1. **Mức 0** — sinh lại bảng, hình và macro số liệu từ các tệp CSV đã có: `python scripts/make_tables.py` và `python scripts/make_numbers.py`.
2. **Mức 1** — sinh lại mọi tệp CSV từ gói kết quả các lượt chạy: giải nén gói rồi chạy `scripts/regenerate_all.sh`; trong Docker, các tệp CSV và bảng sinh ra trùng từng byte với bản đã công bố.
3. **Mức 2** — chạy lại chính SemLoc và các phương pháp đối chứng, phát lại mọi câu trả lời LLM từ bộ đệm, không gọi mạng (`FLX_LLM_OFFLINE=1`).
4. **Mức 3** — chạy mới qua một điểm cuối LLM tương thích OpenAI, với mã lượt chạy mới.

A.1.2 Môi trường

Ảnh Docker (`docker build -t grounded-or-guessed .`) chứa Python 3.12 và toàn bộ phụ thuộc, đặt chế độ không mạng cho LLM; lệnh mặc định chạy bộ kiểm thử đơn vị. Gói kết quả các lượt chạy (đầu ra SemLoc, nhật ký và bộ đệm lượt gọi LLM, bảng băm kiểm tra) được tạo bằng `scripts/export_artifact.py`; công cụ từ chối đóng gói nếu phát hiện khóa truy cập hay địa chỉ thư điện tử.

A.2 BẢNG BỔ SUNG

A.2.1 Phân rã trên nhật ký công bố

Bảng A.1: Phân rã Shapley trên nhật ký công bố của tác giả (dòng lỗi chuẩn gốc)

Cấu hình	Tập con	Chỉ số	SemLoc	Điểm thưởng dòng đoán	Phổ ngữ nghĩa
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	tất cả	Acc@1	42,8	27,8 [22,5; 33,5]	15,0 [12,1; 18,0]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	tất cả	Acc@3	65,6	26,4 [21,9; 31,4]	39,2 [34,2; 44,1]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	thực thi được	Acc@1	57,8	37,6 [31,0; 44,3]	20,3 [16,3; 24,3]
SemLoc, Claude Sonnet 4,6, $T{=}0,8$	thực thi được	Acc@3	77,3	35,7 [30,0; 41,6]	41,6 [36,3; 47,2]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	tất cả	Acc@1	39,6	26,0 [21,1; 31,3]	13,6 [10,8; 16,6]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	tất cả	Acc@3	64,4	24,2 [20,0; 28,9]	40,2 [34,7; 45,4]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	thực thi được	Acc@1	53,5	35,1 [28,6; 42,2]	18,4 [14,7; 22,1]
SemLoc, Gemini 2,5 Pro, $T{=}0,8$	thực thi được	Acc@3	73,0	32,7 [27,3; 38,5]	40,3 [34,8; 45,6]

A.2.2 Kết quả đầy đủ trên ba bộ nhãn

Bảng A.2: RQ2 đầy đủ: mọi biến thể trên ba bộ nhãn (tập test, Gemini 3.8 Flash, 3 lần chạy; Δ so với SemLoc)

Phương pháp	Acc@1	Acc@3	Acc@5	ΔAcc@1 [CI 95%]	ΔAcc@3 [CI 95%]	r_{rb}
Dòng lỗi chuẩn gốc, $n = 200$, 3 lần lặp						
SemLoc (bản phát hành)	36,8±0,8	60,3±2,3	75,0±0,9	–	–	–
bỏ điểm thưởng dòng đoán	29,2±1,4	57,7±2,5	72,7±1,3	-7,7 [-13,9; -1,6]	-2,7 [-4,9; -0,9]	-0,38
chỉ thưởng khi có căn cứ	34,7±1,5	58,2±2,9	72,8±1,5	-2,2 [-4,2; -0,6]	-2,2 [-4,2; -0,6]	-0,94
+ chỉ cổng độ phủ	36,8±0,8	57,7±1,8	62,5±1,3	+0,0 [0,0; 0,0]	-2,7 [-7,7; 1,7]	-0,59
+ chỉ λ·Ochiai	36,8±0,8	61,3±1,9	75,0±0,9	+0,0 [0,0; 0,0]	+1,0 [0,2; 2,3]	+0,54
+ xếp lại bằng CF	35,0±1,0	62,2±2,5	77,7±1,8	-1,8 [-3,8; -0,3]	+1,8 [0,5; 3,5]	+0,36
CovGate (nhóm đề xuất)	36,8±0,8	61,3±1,9	75,0±0,9	+0,0 [0,0; 0,0]	+1,0 [0,2; 2,3]	+0,54
CovGate + xếp lại bằng CF	35,0±1,0	62,5±2,2	77,7±1,8	-1,8 [-3,8; -0,3]	+2,2 [0,9; 3,8]	+0,46
LLM hỏi thẳng (1 prompt)	40,0±0,0	43,0±0,5	43,2±0,8	+3,2 [-0,8; 7,4]	-17,3 [-23,3; -11,1]	-0,51
+ kiểm thử thất bại, không độ phủ (loại bỏ)	44,5±1,0	67,5±1,3	70,3±1,3	+7,7 [3,5; 12,4]	+7,2 [-0,5; 14,6]	+0,16
LLM + độ phủ (1 prompt, nhóm đề xuất)	40,2±0,6	68,2±0,3	70,0±0,9	+3,3 [0,5; 6,5]	+7,8 [0,2; 15,2]	+0,04
SBFL Ochiai	3,0±0,0	15,5±0,0	48,0±0,0	-33,8 [-41,2; -26,8]	-44,8 [-52,8; -36,6]	-0,87
Dòng lỗi chuẩn đã sửa bằng thực thi, $n = 179$, 3 lần lặp						
SemLoc (bản phát hành)	52,3±0,9	75,4±2,0	82,7±1,1	–	–	–
bỏ điểm thưởng dòng đoán	43,2±2,1	70,8±2,3	78,4±1,7	-9,1 [-16,7; -1,9]	-4,7 [-7,9; -2,1]	-0,37
chỉ thưởng khi có căn cứ	48,2±1,7	71,3±2,8	78,6±2,0	-4,1 [-7,2; -1,7]	-4,1 [-7,2; -1,7]	-1,00
+ chỉ cổng độ phủ	52,3±0,9	79,1±2,0	85,1±1,3	+0,0 [0,0; 0,0]	+3,7 [1,5; 6,6]	+1,00
+ chỉ λ·Ochiai	52,3±0,9	79,3±1,7	85,1±1,3	+0,0 [0,0; 0,0]	+3,9 [1,6; 6,8]	+0,85
+ xếp lại bằng CF	49,9±0,9	76,2±1,8	83,4±1,4	-2,4 [-4,8; -0,6]	+0,7 [0,0; 2,1]	-0,35
CovGate (nhóm đề xuất)	52,3±0,9	79,3±1,7	85,1±1,3	+0,0 [0,0; 0,0]	+3,9 [1,6; 6,8]	+0,85
CovGate + xếp lại bằng CF	49,9±0,9	77,1±2,2	85,3±1,2	-2,4 [-4,8; -0,6]	+1,7 [0,5; 3,3]	+0,28
LLM hỏi thẳng (1 prompt)	56,8±0,7	62,8±0,7	62,9±0,9	+4,5 [-0,4; 9,7]	-12,7 [-19,7; -6,2]	-0,28
+ kiểm thử thất bại, không độ phủ (loại bỏ)	62,6±1,1	89,9±1,0	92,0±0,9	+10,2 [5,3; 16,0]	+14,5 [8,5; 20,8]	+0,64
LLM + độ phủ (1 prompt, nhóm đề xuất)	57,2±0,9	92,0±0,3	93,7±0,9	+4,8 [1,5; 8,7]	+16,6 [10,5; 22,8]	+0,65
SBFL Ochiai	3,9±0,0	19,0±0,0	67,6±0,0	-48,4 [-58,6; -38,3]	-56,4 [-67,0; -45,4]	-0,85
Dòng lỗi chuẩn đã rà soát, $n = 200$, 3 lần lặp						
SemLoc (bản phát hành)	83,0±1,3	90,0±1,8	91,8±1,6	–	–	–
bỏ điểm thưởng dòng đoán	56,8±2,5	83,5±2,8	85,7±2,8	-26,2 [-33,1; -19,6]	-6,5 [-9,8; -3,6]	-0,80
chỉ thưởng khi có căn cứ	76,8±2,2	84,0±3,0	85,7±2,8	-6,2 [-9,3; -3,4]	-6,0 [-9,2; -3,2]	-0,98
+ chỉ cổng độ phủ	83,0±1,3	91,2±1,9	92,0±1,8	+0,0 [0,0; 0,0]	+1,2 [0,0; 2,8]	+1,00
+ chỉ λ·Ochiai	83,0±1,3	91,2±1,9	92,0±1,8	+0,0 [0,0; 0,0]	+1,2 [0,0; 2,8]	+0,67
+ xếp lại bằng CF	78,7±1,0	90,3±1,8	91,8±1,3	-4,3 [-7,2; -2,0]	+0,3 [0,0; 1,1]	-0,94
CovGate (nhóm đề xuất)	83,0±1,3	91,2±1,9	92,0±1,8	+0,0 [0,0; 0,0]	+1,2 [0,0; 2,8]	+0,67
CovGate + xếp lại bằng CF	78,7±1,0	90,7±2,0	91,8±1,3	-4,3 [-7,2; -2,0]	+0,7 [0,0; 1,6]	-0,69
LLM hỏi thẳng (1 prompt)	87,3±0,6	90,8±0,3	91,0±0,5	+4,3 [0,3; 8,6]	+0,8 [-2,4; 4,0]	+0,28
+ kiểm thử thất bại, không độ phủ (loại bỏ)	95,5±0,9	100,0±0,0	100,0±0,0	+12,5 [8,0; 17,5]	+10,0 [6,3; 14,3]	+0,94
LLM + độ phủ (1 prompt, nhóm đề xuất)	92,2±0,8	100,0±0,0	100,0±0,0	+9,2 [5,4; 13,2]	+10,0 [6,3; 14,3]	+0,92
SBFL Ochiai	11,0±0,0	28,0±0,0	84,0±0,0	-72,0 [-80,4; -62,6]	-62,0 [-71,2; -52,1]	-0,96

A.3 CÁC PROMPT

A.3.1 LLM + kiểm thử thất bại (phương pháp đề xuất) và biến thể có độ phủ

Thông điệp hệ thống yêu cầu trả lời đúng định dạng JSON. Biến thể có độ phủ thêm dòng giải thích chú thích (đánh dấu `[cov]` dưới đây) và chú thích `[f=…/p=…]` ở cuối mỗi dòng mã; mọi phần khác giống hệt.

```
You are a fault-localization assistant. Reply with JSON only:
{"lines": [<line numbers, most suspicious first, at most 5>]}

The program below has exactly one faulty line. <n_fail> tests fail and
   <n_pass> pass.
[cov] Each line is annotated with [f=<number of failing tests that
   execute it>/p=<number of passing tests>].
<numbered source>
Failing tests:
- <test name>: <first error line>
Return the five most suspicious line numbers, most suspicious first.
```

A.3.2 Đề xuất bản sửa (bộ phán định nhãn và đề xuất rồi kiểm chứng)

```
You fix one-line bugs in Python. Reply with JSON only:
{"kind": "replace" | "insert_after", "code": "<one line of Python, no
   indentation>"}

Program (line-numbered): <numbered source>
Failing tests: <pytest failure summary>
Assume the bug is at line <n>. Give the single line that should replace
    line <n> (or, if code is missing,
```

```
the line to insert after line <n>) so that all tests pass. Make the
    smallest possible change (ideally one
or two tokens) and keep everything else identical. Attempt <k>.
```

A.4 QUY TRÌNH DUYỆT NHÃN

Mỗi thành viên nhận một tệp gồm các ca được giao (`docs/analysis/gt-review/thanh-vien/`). Mỗi ca có liên kết tới chương trình và kiểm thử, đoạn mã quanh các dòng ứng viên, và nhận định của LLM để tham khảo; các dòng ứng viên được ký hiệu G (nhãn gốc), S (dòng bộ phán định dời tới) và F (dòng LLM đề xuất). Người duyệt ghi vào cột `answer` một hay nhiều chữ (ví dụ `G+F` cho lỗi hai dòng) hoặc một số dòng khác, và có thể ghi `?` nếu không chắc. Bộ nhãn đã rà soát được dựng tự động từ các tệp này (`scripts/build_reviewed_gt.py`); các ca trưởng nhóm sửa được liệt kê trong mã kèm lý do.

Tài liệu tham khảo

[1] Z. Yang, H. Zhu, Q. Zhang, R. Gupta, and A. Kundu, “Semloc: Structured grounding of free-form llm reasoning for fault localization,” 2026. [Online]. Available: https://arxiv.org/abs/2603.29109

[2] H. Yang, Y. Peng, and Z. Ding, “Effiholmes: Differential profiling-guided repository level time inefficiency fix localization,” 2026. [Online]. Available: https://arxiv.org/abs/2608.03558

[3] M. Medeiros, U. Kulesza, C. Treude, D. Lucena, R. Gomes, R. Coelho, E. Adachi, and R. Bonifacio, “Integrating crash report mining and llms for bug localization and repair: An industrial report,” 2026. [Online]. Available: https://arxiv.org/abs/2609.04483

[4] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.

[5] Stack Overflow, “2025 developer survey,” 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[6] JetBrains, “The state of developer ecosystem 2025,” 2025. [Online]. Available: https://blog.jetbrains.com/research/2025/10/state-of-developer-ecosystem-2025/

[7] T. Xiao, C. Treude, H. Hata, and K. Matsumoto, “DevGPT: Studying developer-ChatGPT conversations,” in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR), Mining Challenge*, 2024. [Online]. Available: https://arxiv.org/abs/2309.03914

[8] Y. Wu, Z. Li, J. M. Zhang, M. Papadakis, M. Harman, and Y. Liu, “Large language models in fault localisation,” 2023. [Online]. Available: https://arxiv.org/abs/2308.15276

[9] S. Kang, G. An, and S. Yoo, “A quantitative and qualitative evaluation of llm-based explainable fault localization,” 2023. [Online]. Available: https:

//arxiv.org/abs/2308.05487

[10] J. A. Jones, M. J. Harrold, and J. Stasko, “Visualization of test information to assist fault localization,” in *Proceedings of the 24th international conference on Software engineering - ICSE ’02*, ser. ICSE ’02. ACM Press, 2002, p. 467. [Online]. Available: http://dx.doi.org/10.1145/581396.581397

[11] R. Abreu, P. Zoeteweij, R. Golsteijn, and A. J. van Gemund, “A practical evaluation of spectrum-based fault localization,” *Journal of Systems and Software*, vol. 82, no. 11, p. 1780–1792, Nov. 2009. [Online]. Available: http://dx.doi.org/10.1016/j.jss.2009.06.035

[12] W. E. Wong, R. Gao, Y. Li, R. Abreu, and F. Wotawa, “A survey on software fault localization,” *IEEE Transactions on Software Engineering*, vol. 42, no. 8, p. 707–740, Aug. 2016. [Online]. Available: http://dx.doi.org/10.1109/TSE.2016.2521368

[13] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh, “Bugsinpy: a database of existing bugs in python programs to enable controlled testing and debugging studies,” in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE ’20. ACM, Nov. 2020, p. 1556–1560. [Online]. Available: http://dx.doi.org/10.1145/3368089.3417943

[14] W. E. Wong, V. Debroy, R. Gao, and Y. Li, “The dstar method for effective software fault localization,” *IEEE Transactions on Reliability*, vol. 63, no. 1, p. 290–308, Mar. 2014. [Online]. Available: http://dx.doi.org/10.1109/TR.2013.2285319

[15] H. A. de Souza, M. L. Chaim, and F. Kon, “Spectrum-based software fault localization: A survey of techniques, advances, and challenges,” 2016. [Online]. Available: https://arxiv.org/abs/1607.04347

[16] M. Papadakis and Y. Le Traon, “Metallaxis-fl: mutation-based fault localization,”

Software Testing, Verification and Reliability, vol. 25, no. 5-7, p. 605–628, Sep. 2013. [Online]. Available: http://dx.doi.org/10.1002/stvr.1509

[17] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*. IEEE, Mar. 2014, p. 153–162. [Online]. Available: http://dx.doi.org/10.1109/ICST.2014.28

[18] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, p. 183–200, Feb. 2002. [Online]. Available: http://dx.doi.org/10.1109/32.988498

[19] A. Zeller, "Isolating cause-effect chains from computer programs," *ACM SIGSOFT Software Engineering Notes*, vol. 27, no. 6, p. 1–10, Nov. 2002. [Online]. Available: http://dx.doi.org/10.1145/605466.605468

[20] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, p. 35–45, Dec. 2007. [Online]. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015

[21] Y. Qin, S. Wang, Y. Lou, J. Dong, K. Wang, X. Li, and X. Mao, "Agentfl: Scaling llm-based fault localization to project-level context," 2024. [Online]. Available: https://arxiv.org/abs/2403.16362

[22] C. Xu, Z. Liu, X. Ren, G. Zhang, M. Liang, and D. Lo, "Flexfl: Flexible and effective fault localization with open-source large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.10714

[23] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024. [Online]. Available: https://arxiv.org/abs/2407.01489

[24] C. Parnin and A. Orso, "Are automated debugging techniques actually helping

programmers?" in *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2011, pp. 199–209.

[25] P. S. Kochhar, X. Xia, D. Lo, and S. Li, "Practitioners' expectations on automated fault localization," in *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2016, pp. 165–176.

[26] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA '14. ACM, 2014, p. 437–440. [Online]. Available: http://dx.doi.org/10.1145/2610384.2628055

[27] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, ser. SPLASH '17. ACM, Oct. 2017, p. 55–56. [Online]. Available: http://dx.doi.org/10.1145/3135932.3135941

[28] S. Pearson, J. Campos, R. Just, G. Fraser, R. Abreu, M. D. Ernst, D. Pang, and B. Keller, "Evaluating and improving fault localization," in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, May 2017, p. 609–620. [Online]. Available: http://dx.doi.org/10.1109/ICSE.2017.62

[29] C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive label errors in test sets destabilize machine learning benchmarks," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2103.14749

[30] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*.

ACM, 2015, pp. 24–36.

[31] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, “Is the cure worse than the disease? overfitting in automated program repair,” in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2015, pp. 532–543.

[32] J. Sallou, T. Durieux, and A. Panichella, “Breaking the silence: the threats of using llms in software engineering,” 2023. [Online]. Available: https: //arxiv.org/abs/2312.08055

[33] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, “Are large language models memorizing bug benchmarks?” 2024. [Online]. Available: https://arxiv.org/abs/2411.13323

[34] C. Xu, S. Guan, D. Greene, and M.-T. Kechadi, “Benchmark data contamination of large language models: A survey,” 2024. [Online]. Available: https://arxiv.org/abs/2406.04244